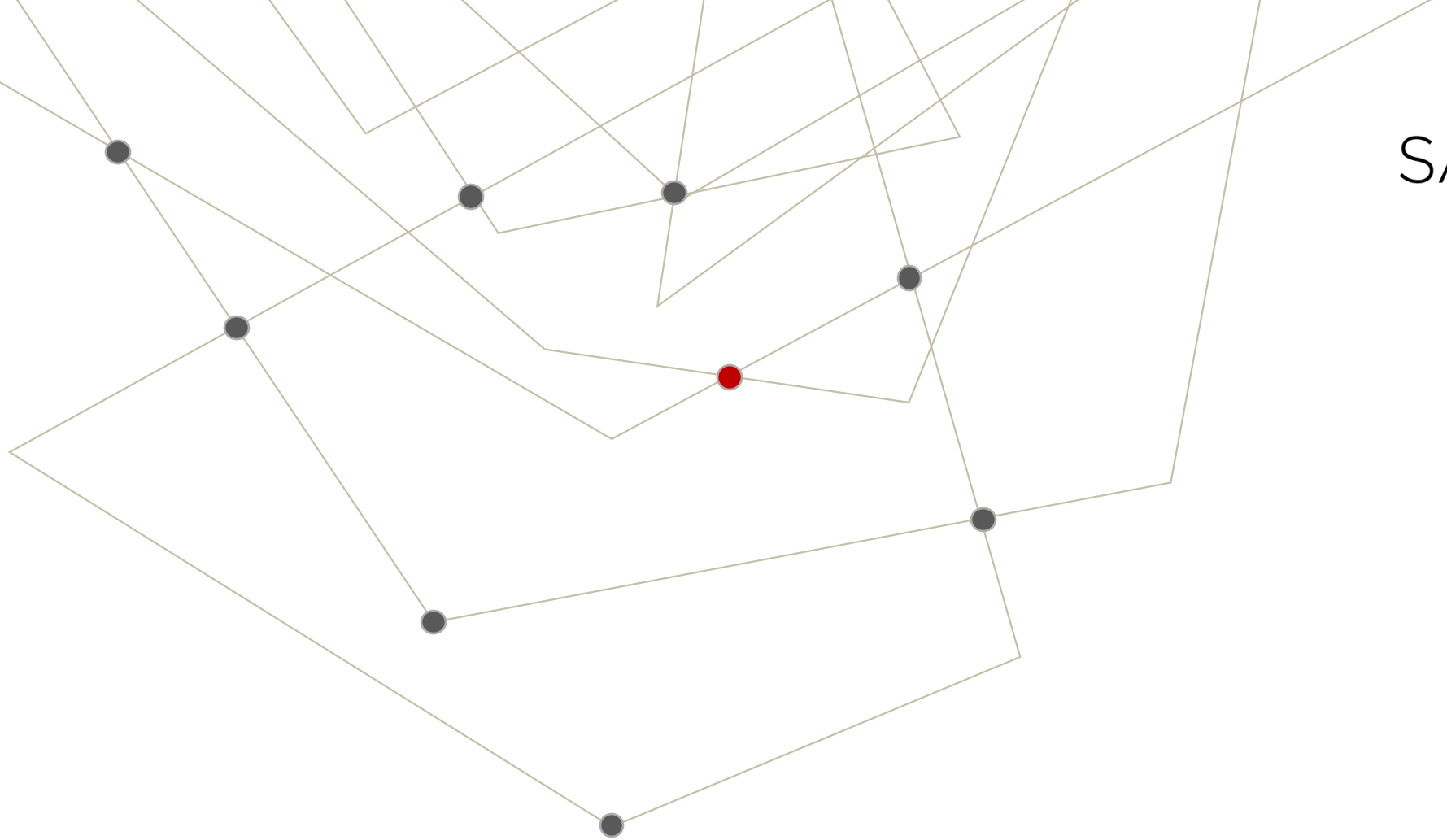
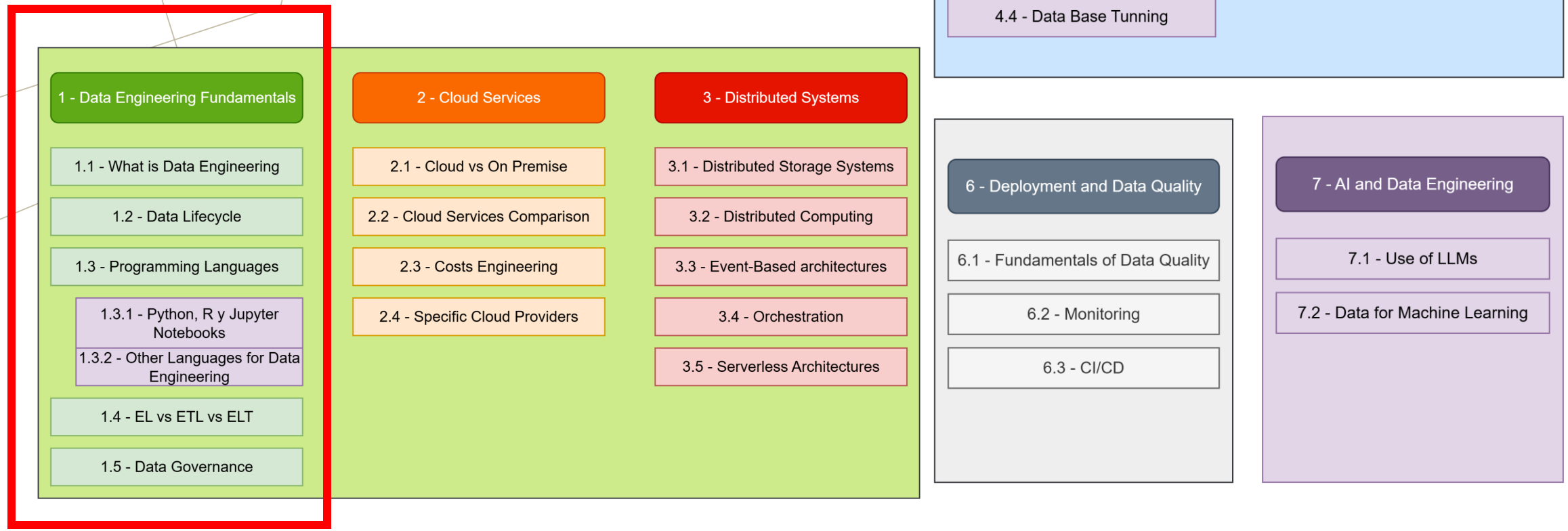


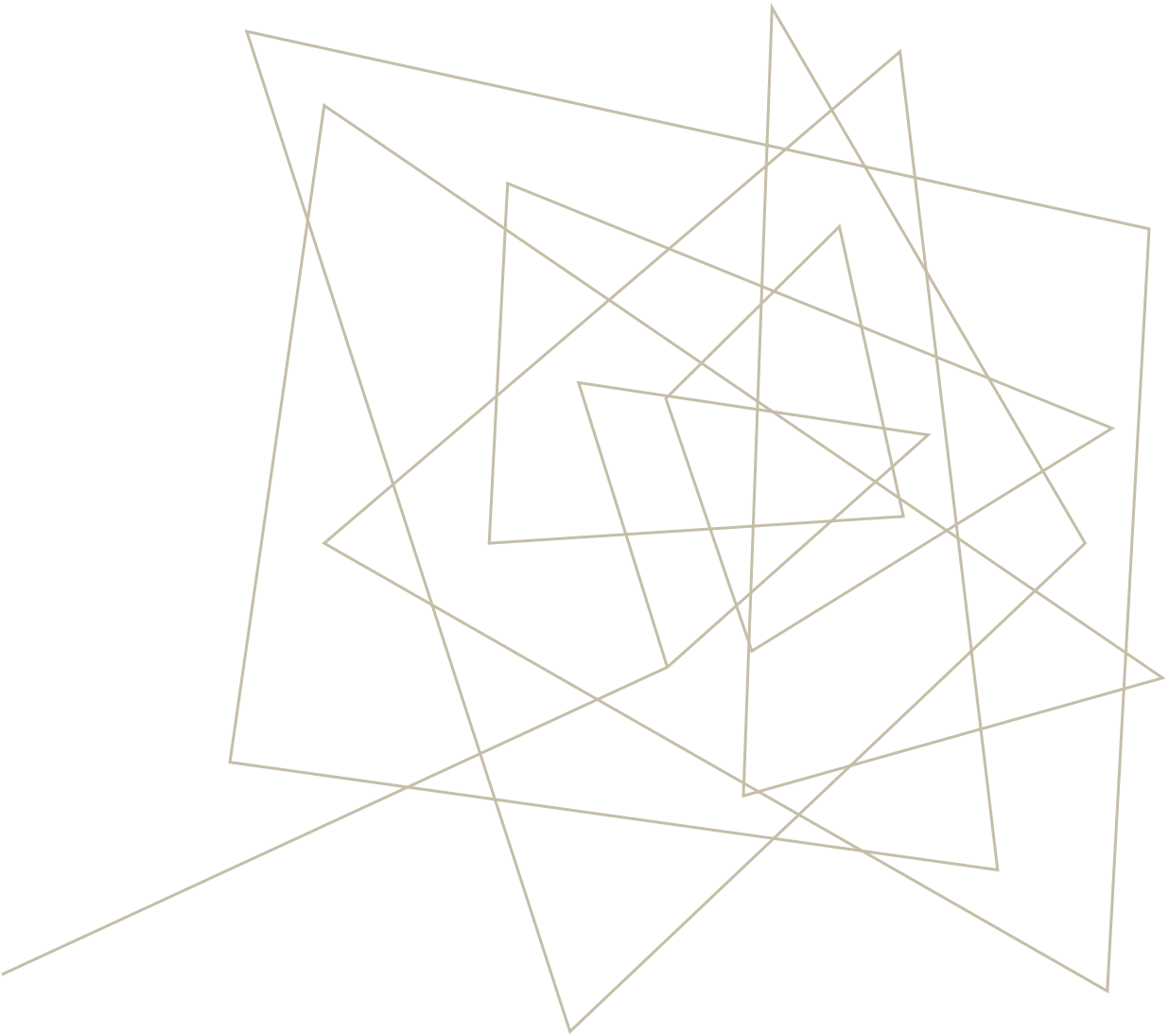
universidad SANJORGE
GRUPO SANVALERO

Artificial Intelligence Degree at USJ

DATA ENGINEERING: FUNDAMENTALS

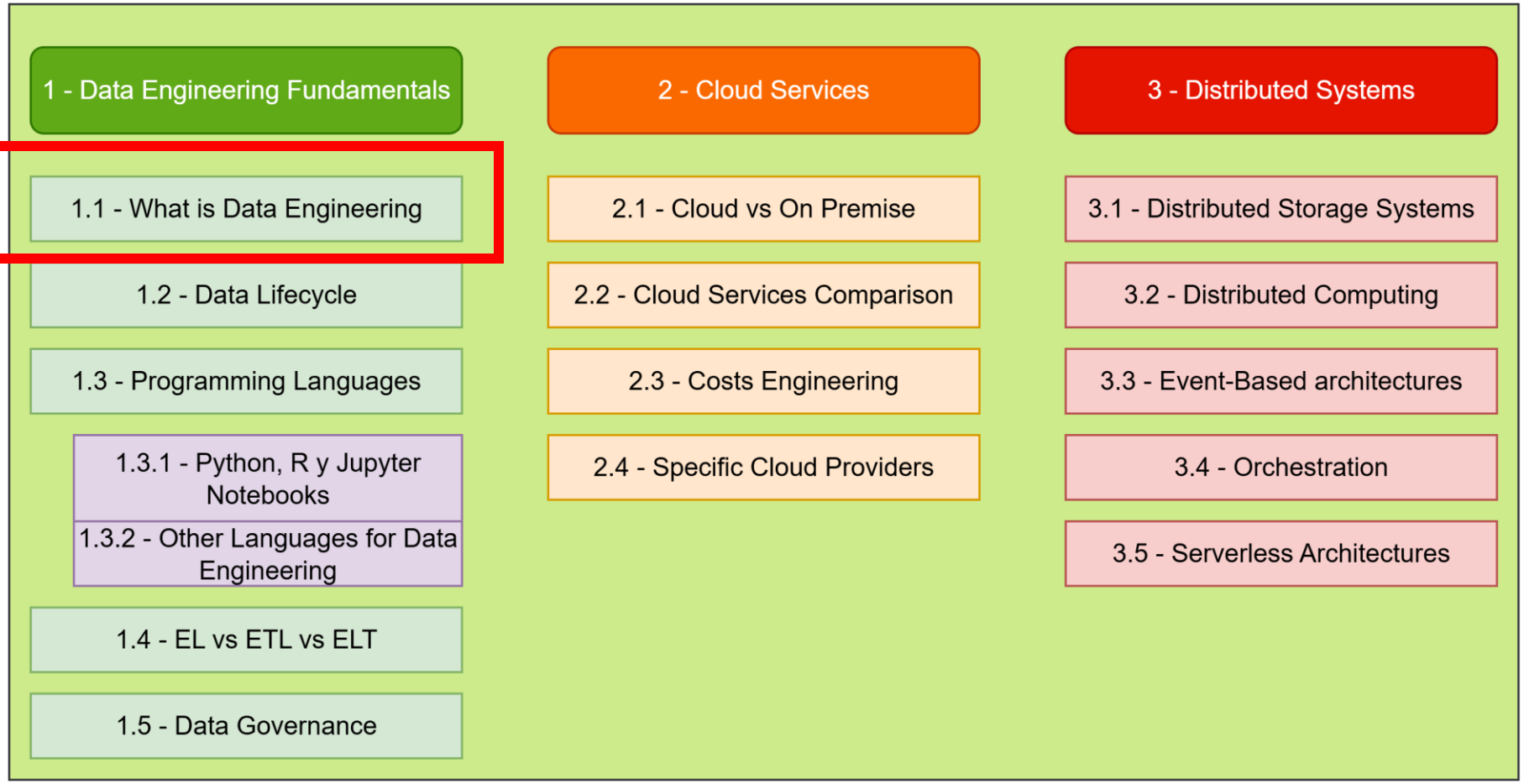
SUBJECT CURRICULUM





WHAT IS DATA ENGINEERING?

SUBJECT CURRICULUM



WHAT IS DATA ENGINEERING

- **Concept:** Raw data is valuable but not usable in its native state.
- Data is messy, voluminous, and needs to be processed to become a valuable product.
- Data Engineers build the structure that make this possible.
- Without this infrastructure, data science and analytics cannot happen at scale.

WHAT IS DATA ENGINEERING

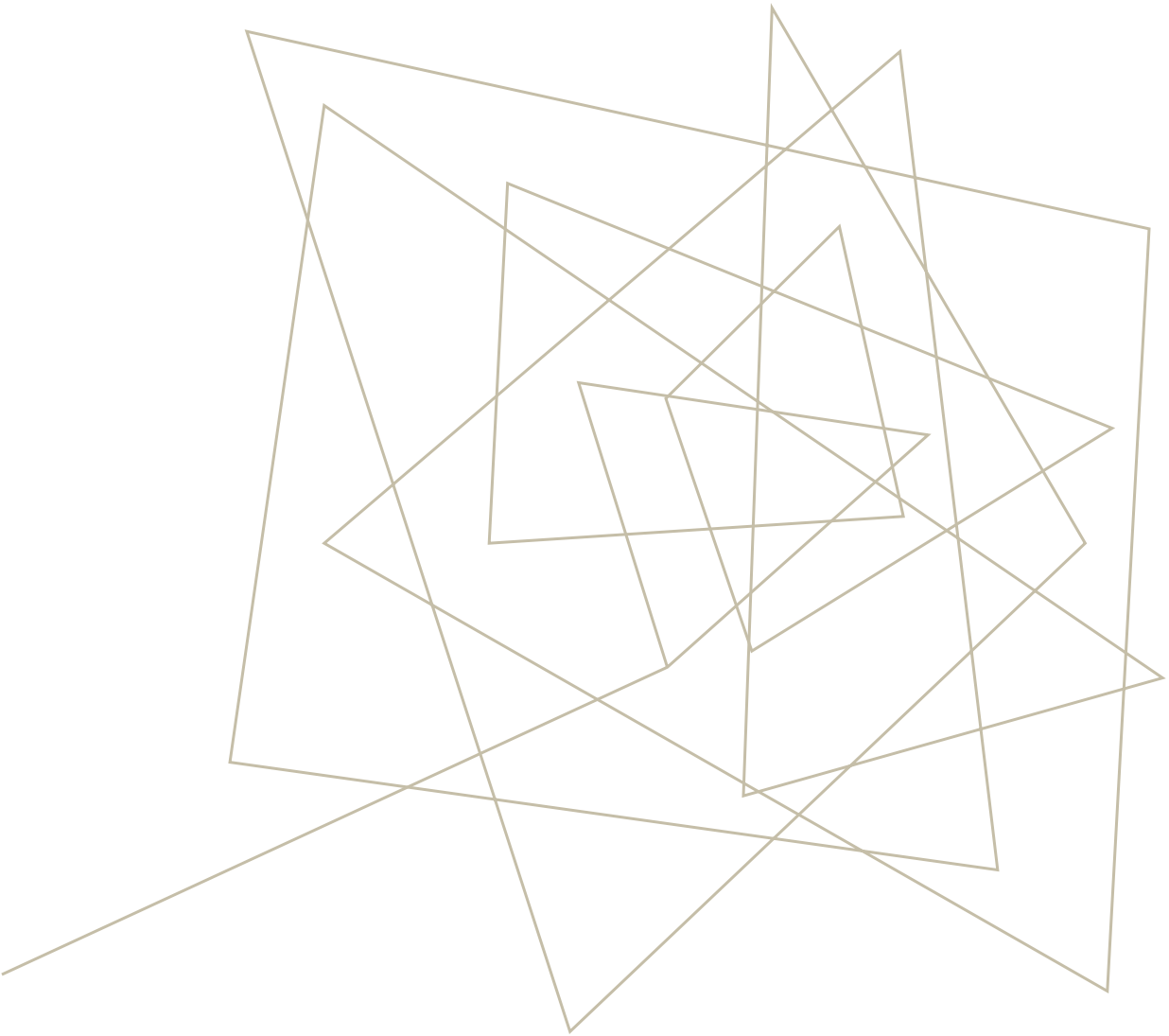
Role	Goal	Key Question
Data Engineer	Build the System	How do we reliably move and prepare data?
Data Scientist	Predict the Future	What can we learn/predict from this data?
Data Analyst	Explain the Past	What happened and why?

WHAT IS DATA ENGINEERING

Watch this video on youtube: https://www.youtube.com/watch?v=hTjo-QVWcK0

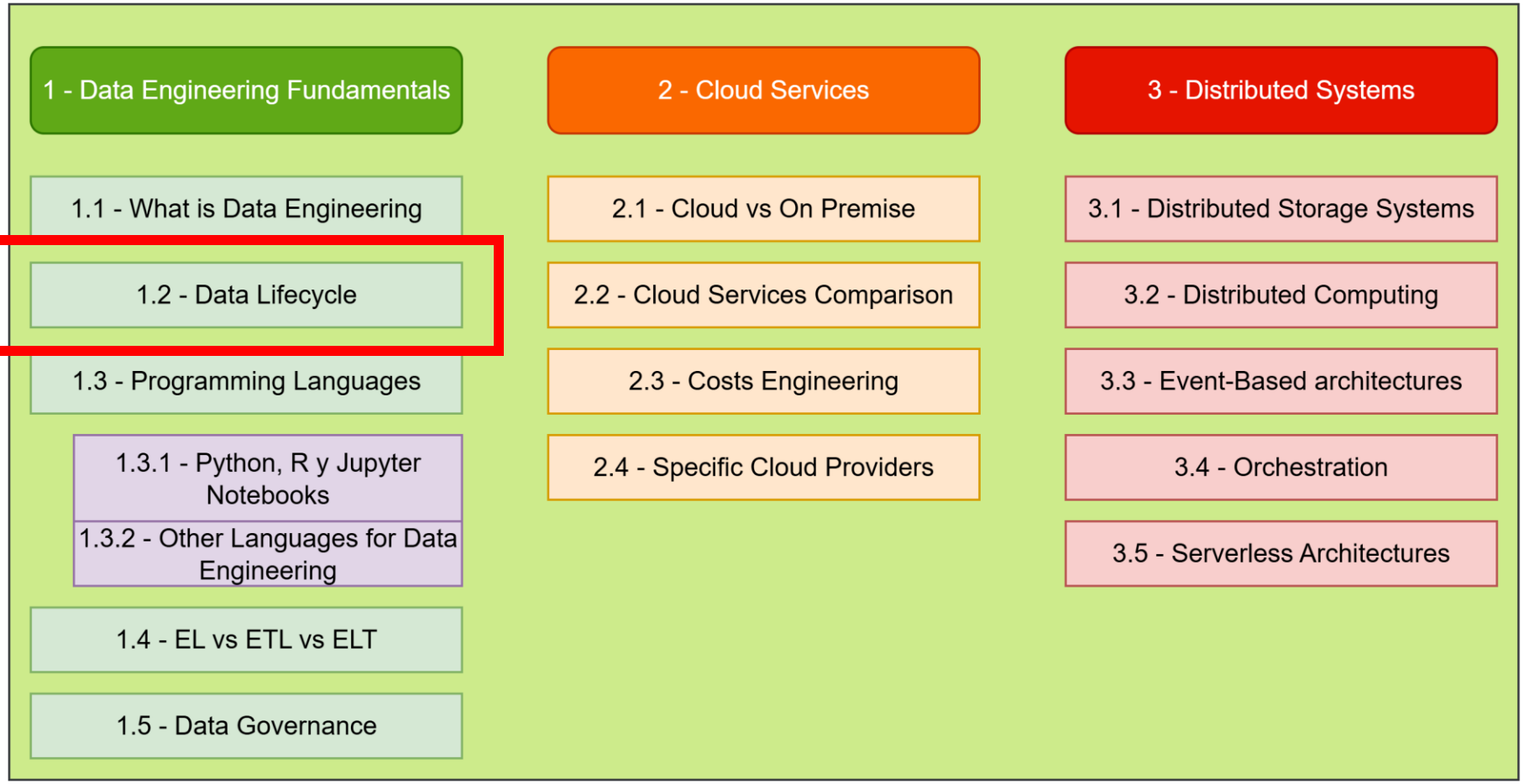
Answer these questions based on the video (and add anything you want):

- What are the responsibilities of a data engineer?
- What are the key tasks a data engineer has?
- What are the key skills?
- You are a CEO of a company. Think of situations that you will need to ask a Data Engineer, A Data Analyst, or a Data Scientist.
- Can these data roles overlap?



DATA LIFECYCLE

SUBJECT CURRICULUM



DATA LIFECYCLE

- Data follows a predictable path from its creation to its final use. This path provides a crucial framework for all data engineering work.

GENERATION

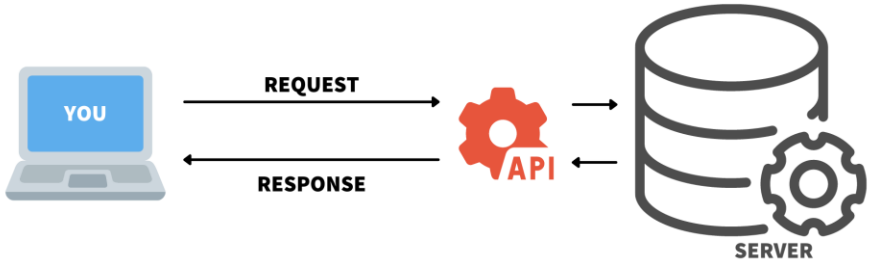
- **Data** are a collection of discrete or continuous values that convey information, describing the quantity, quality, fact, statistics, other basic units of meaning, or simply sequences of symbols that may be further interpreted formally.

Can you think about how data is generated?

RETRIEVAL

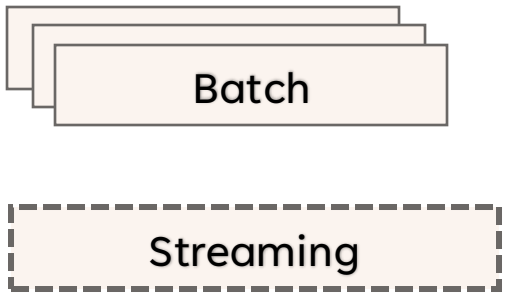
Gathering data can be accomplished through a primary source (the researcher is the first person to obtain the data) or a secondary source (the researcher obtains the data that has already been collected by other sources, such as data disseminated in a scientific journal).

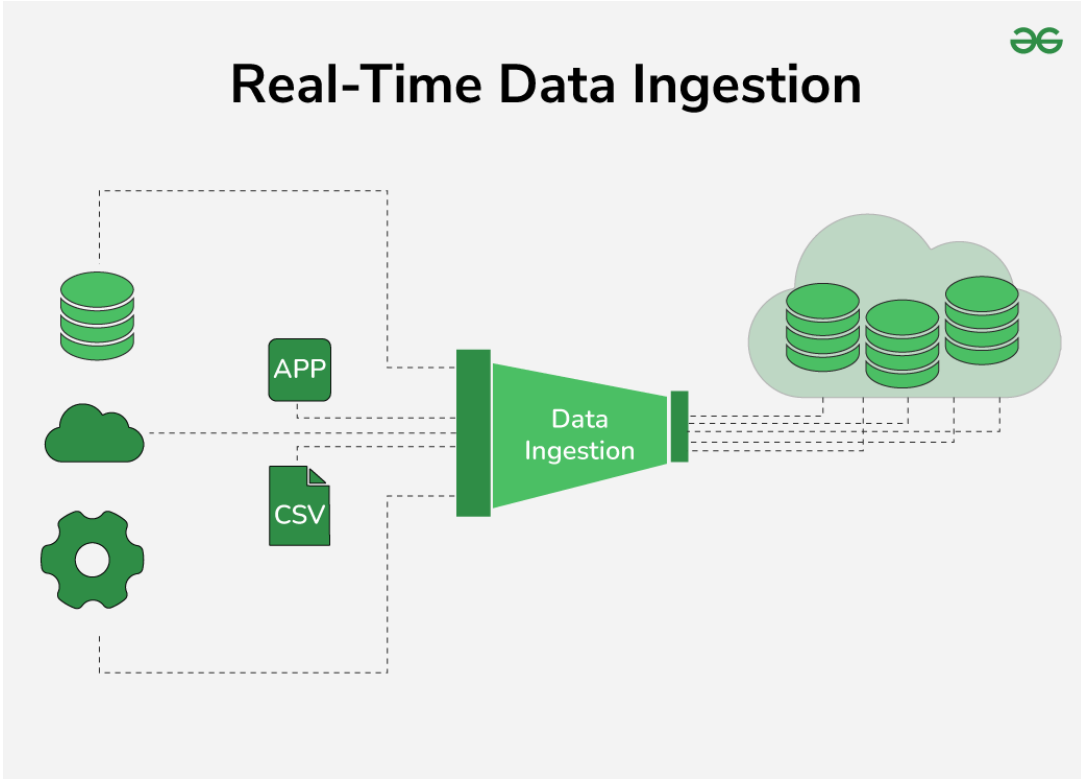
Data source	Owned by	Accuracy	Use case	Privacy risk
First-party	The business	High	Personalization, retargeting	Low
Second-party	Partner	Moderate	Partnership campaigns	Moderate
Third-party	External entity	Low	Broad targeting	High



INGESTION

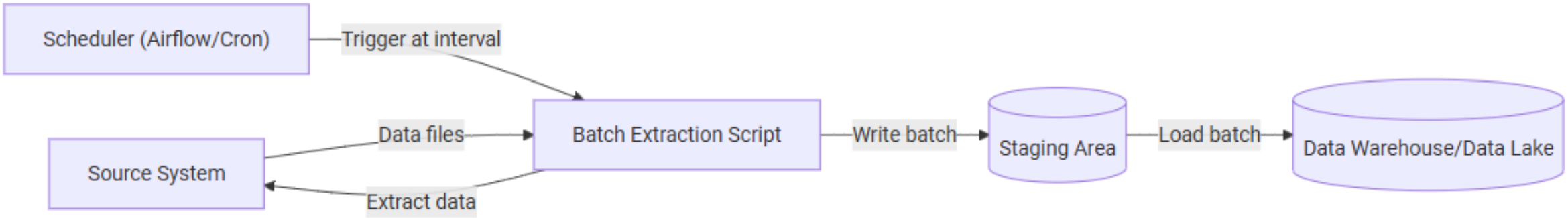
- The process of moving data from its source into a centralized storage system.





INGESTION

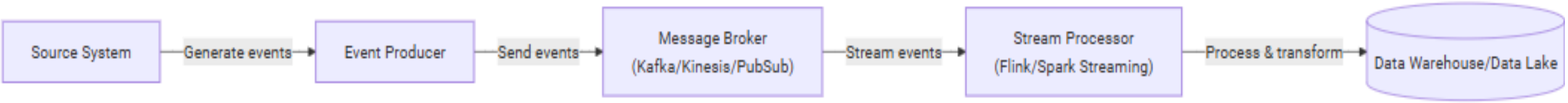
- Higher latency (minutes to hours)
- More efficient for large volumes
- Easiest to implement and debug
- Lower infrastructure costs



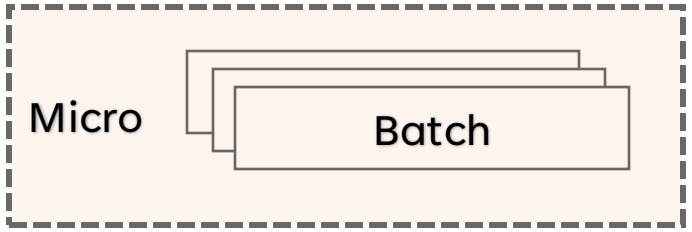
INGESTION

Streaming

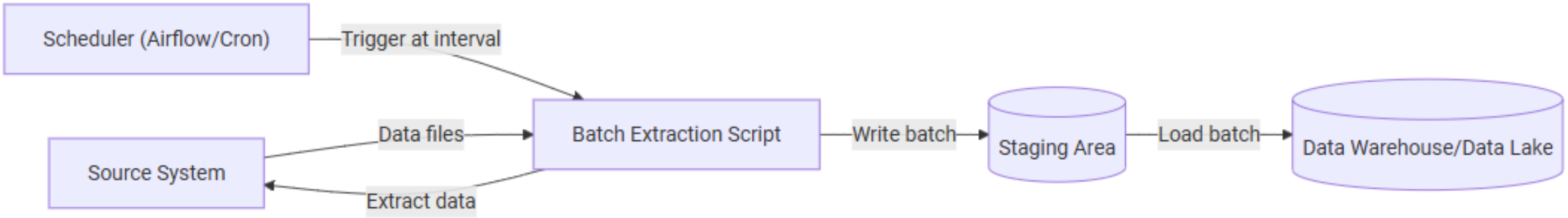
- Low latency (seconds to milliseconds)
- More complex to implement
- Higher infrastructure costs
- Enables real-time analytics



INGESTION



- **Near** real-time processing (typically 5-15 minutes)
- Balances latency and efficiency
- Easier than true streaming
- Good for most use cases



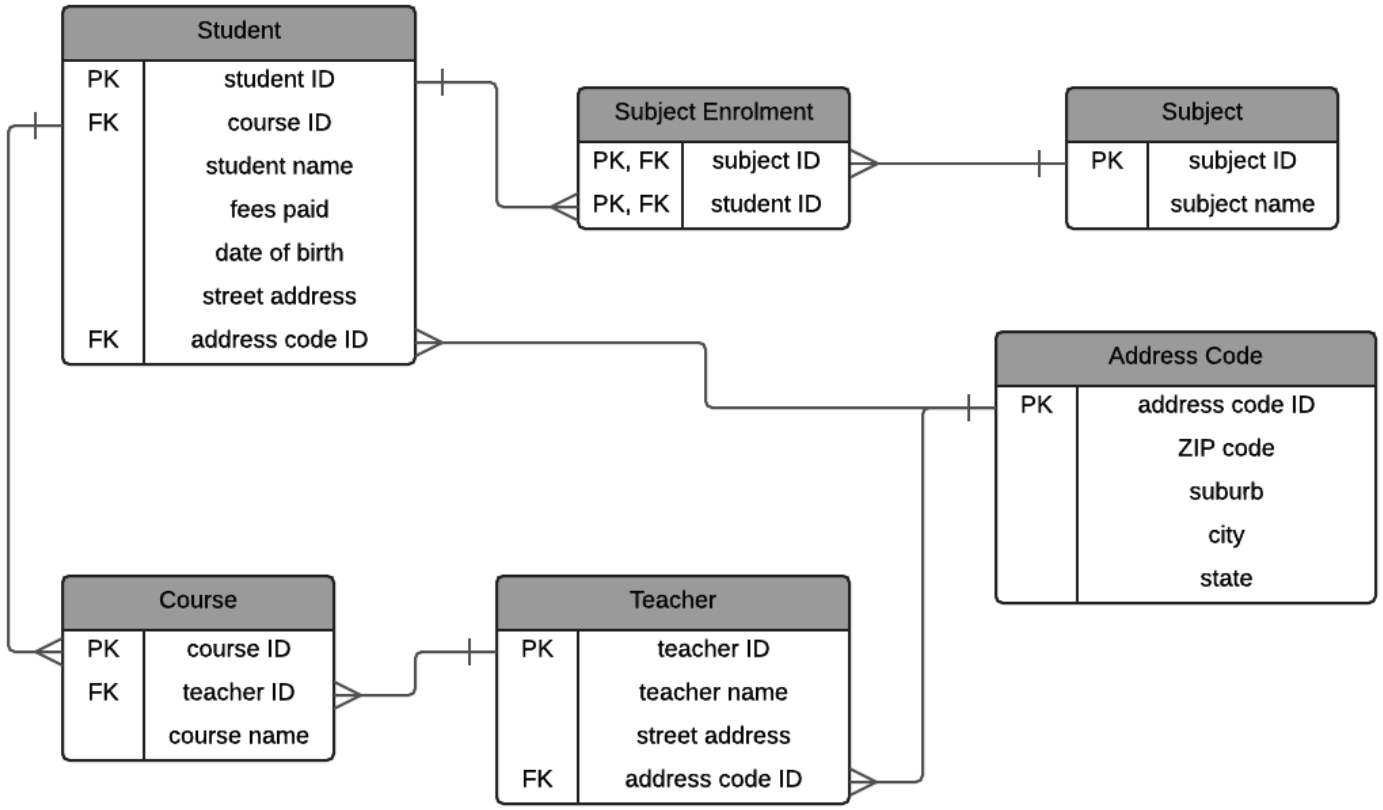
STORAGE

- Persistently storing the ingested data for future use.
- Structured or Unstructured
- Normalized or Denormalized

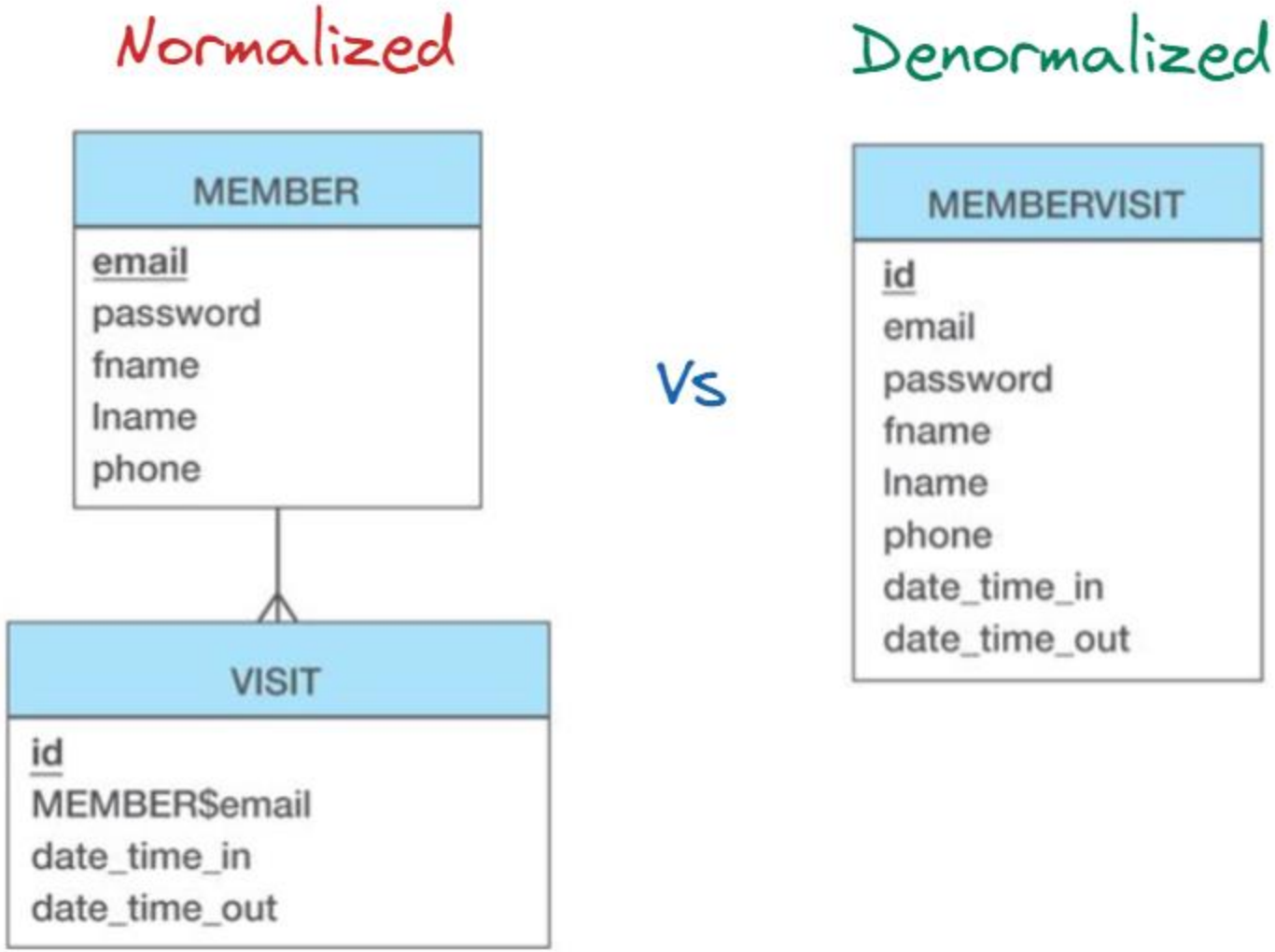
Table: customers

customer_id	first_name	last_name	phone	country
1	John	Doe	817-646-8833	USA
2	Robert	Luna	412-862-0502	USA
3	David	Robinson	208-340-7906	UK
4	John	Reinhardt	307-242-6285	UK
5	Betty	Taylor	806-749-2958	UAE

STORAGE: NORMALIZED DATA



STORAGE: DENORMALIZATION

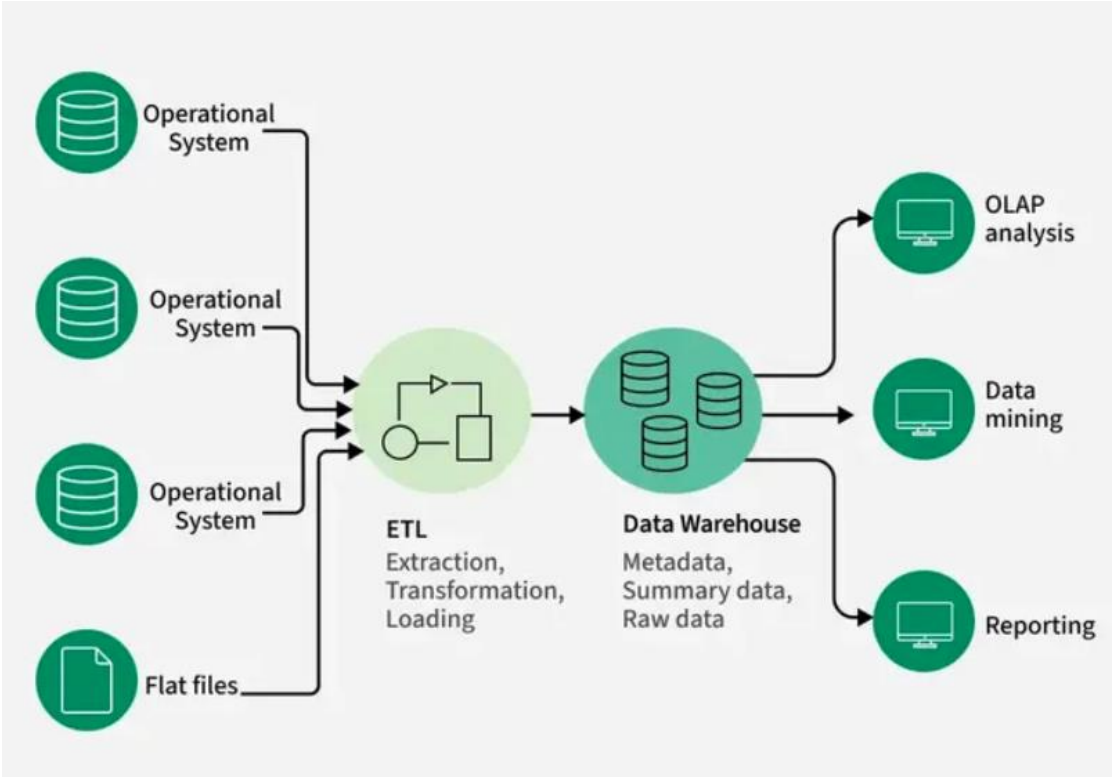


PROCESSING

- Clean
- Validate
- Aggregate
- Enrich
- Transform

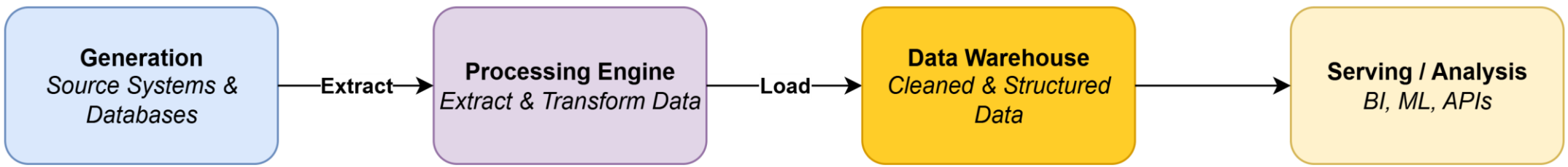
SERVING

- Data Warehouse
- Data Lakehouse
- Data Mesh
- API
- Flat files by email?
- Dashboards?

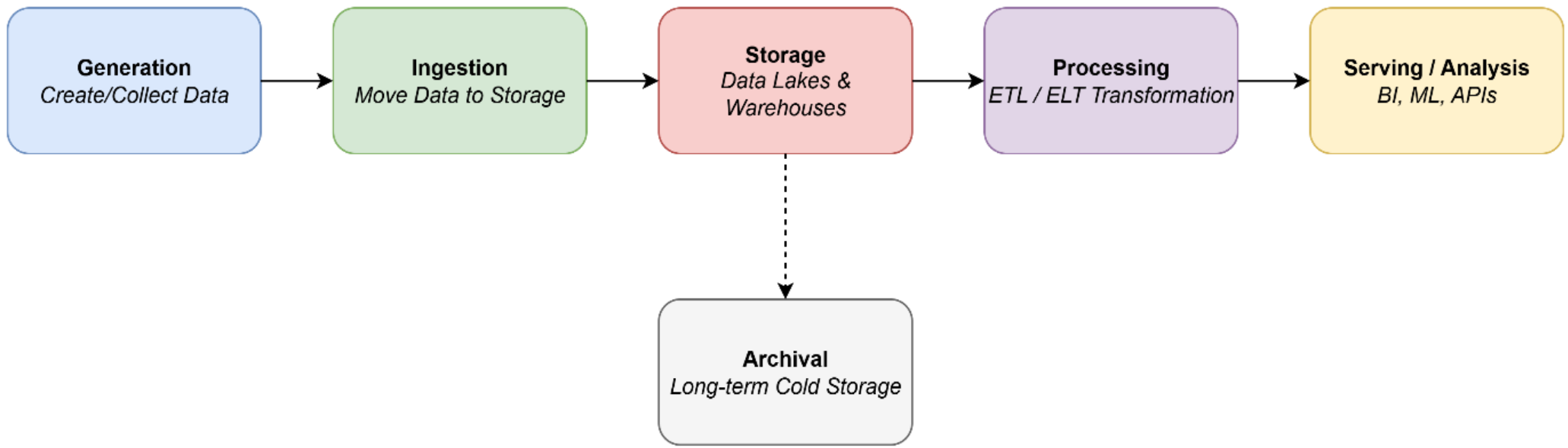


Is ‘Data’ a product?

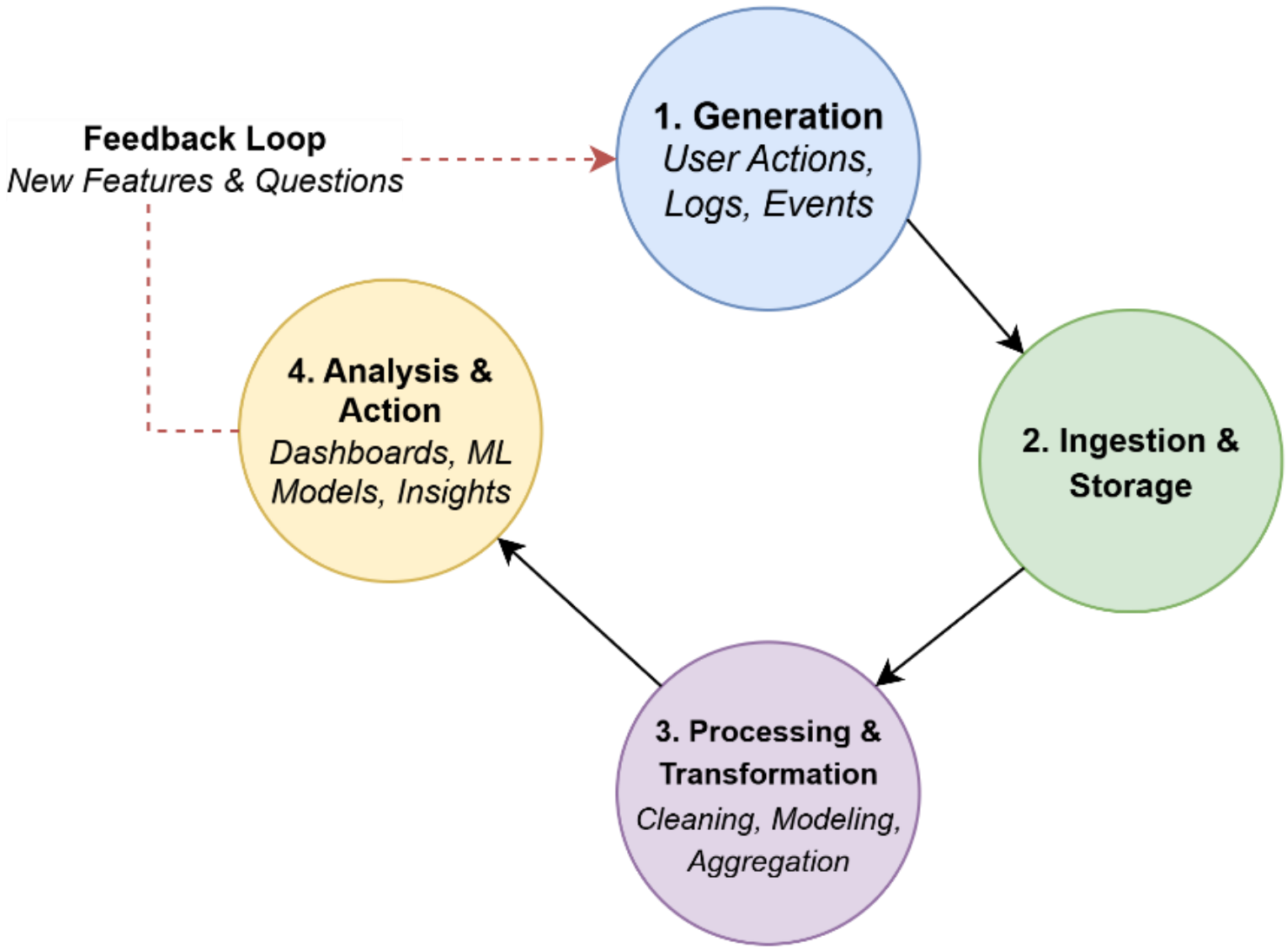
Data Lifecycle with Data Warehouse (Classic ETL Pattern)



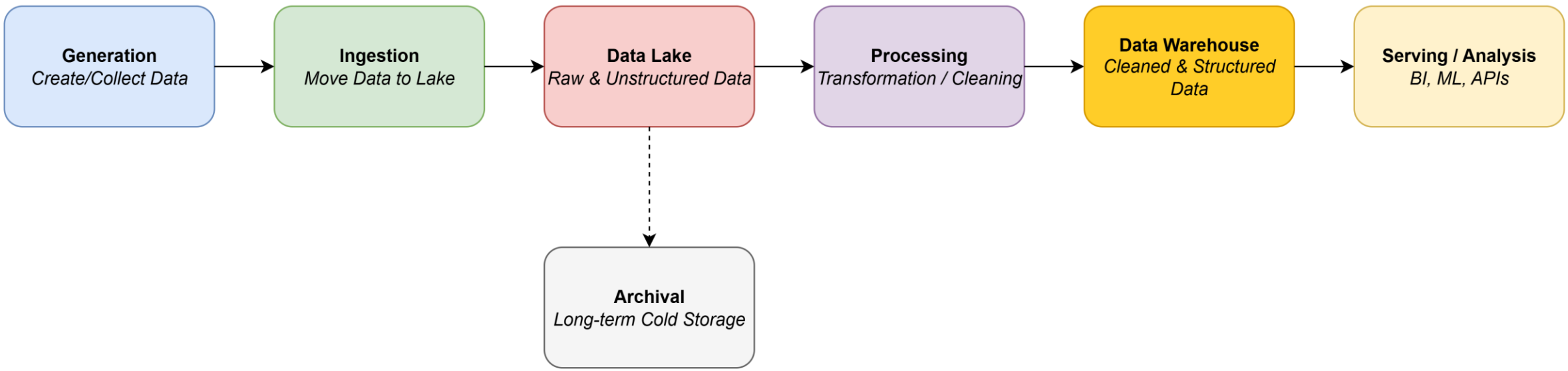
The Linear Data Lifecycle

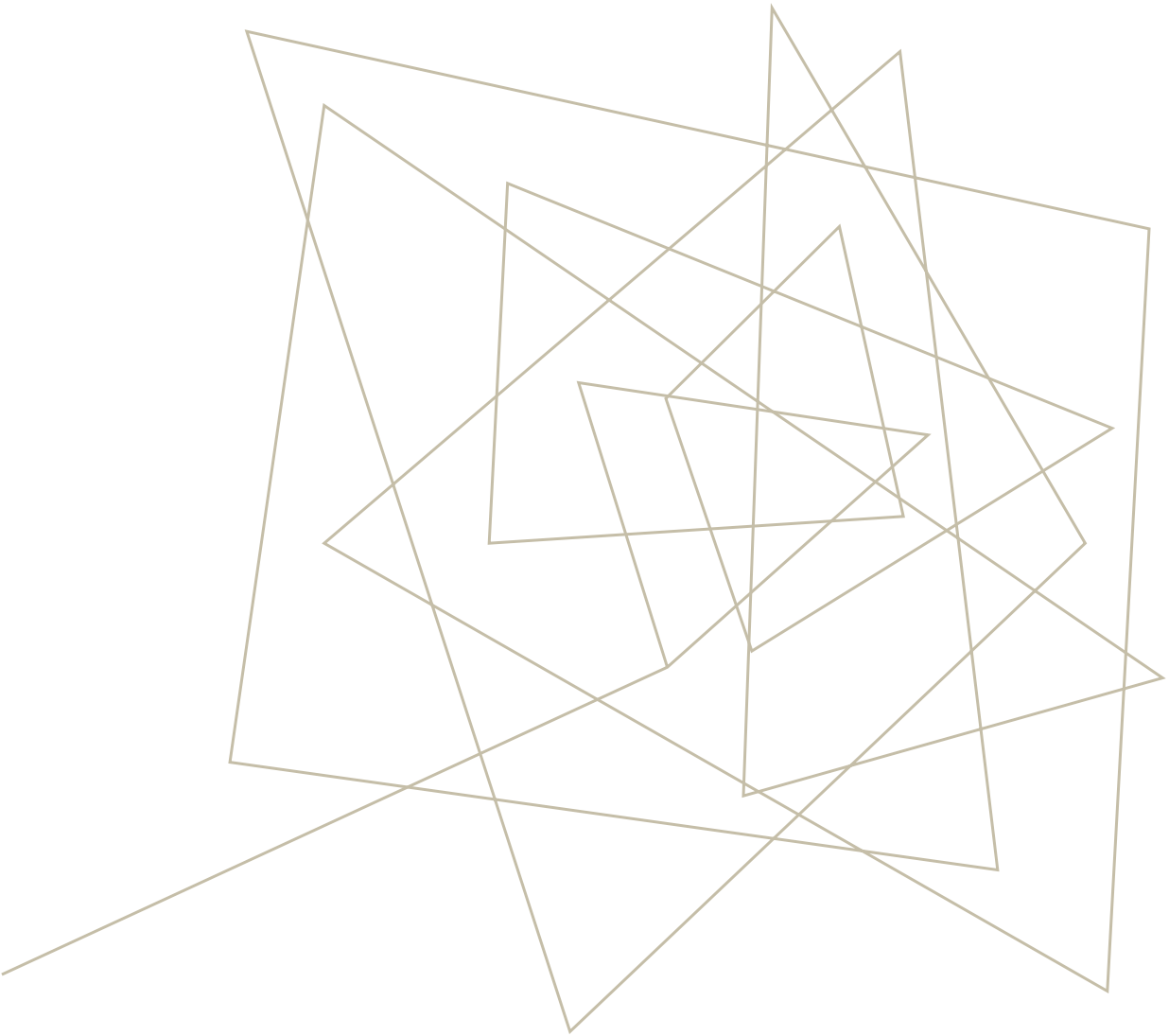


The Cyclical (Feedback-Driven) Data Lifecycle



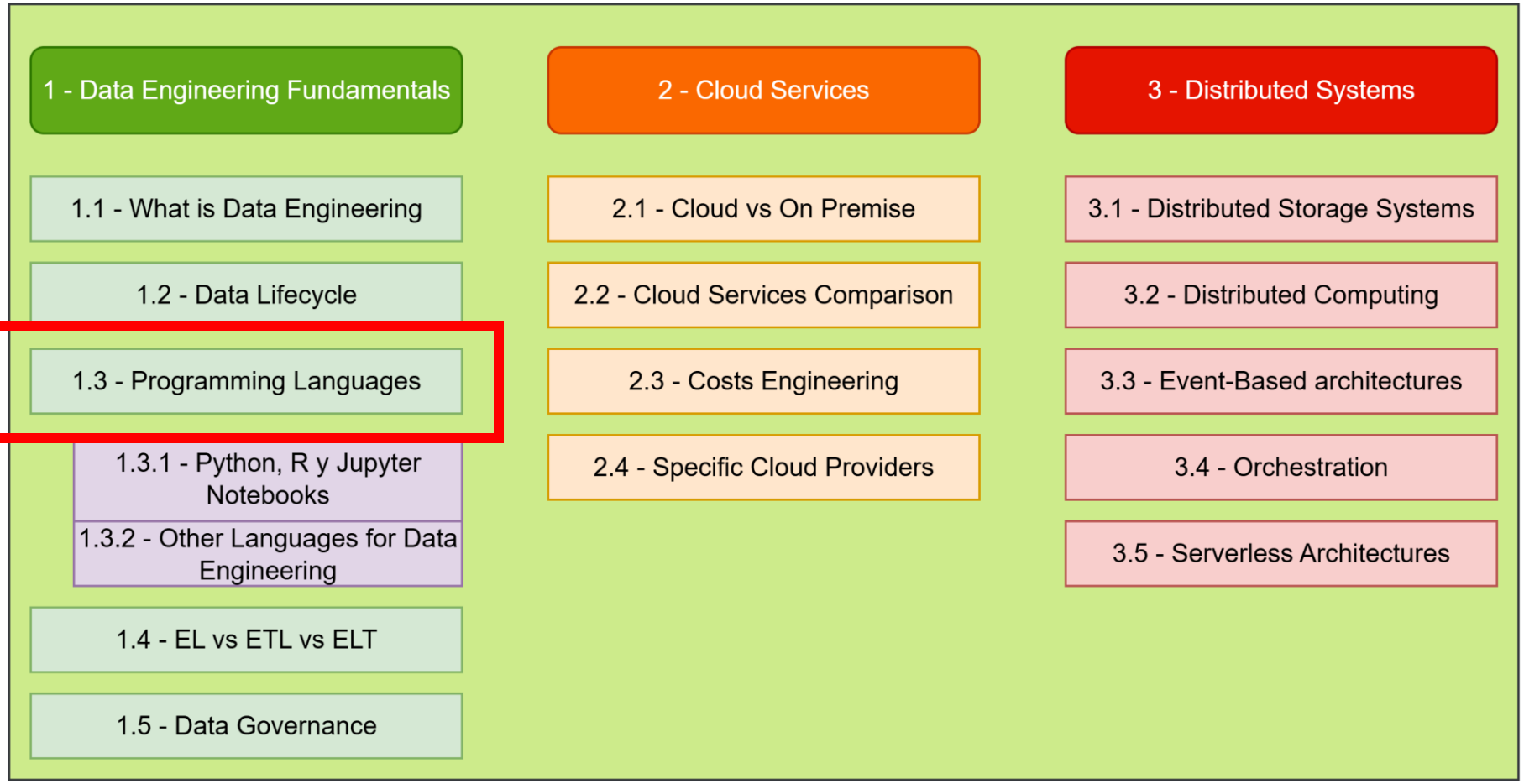
Data Lifecycle with Data Warehouse





PROGRAMMING LANGUAGES

SUBJECT CURRICULUM




python


SQL


jupyter


Scala


PySpark


R

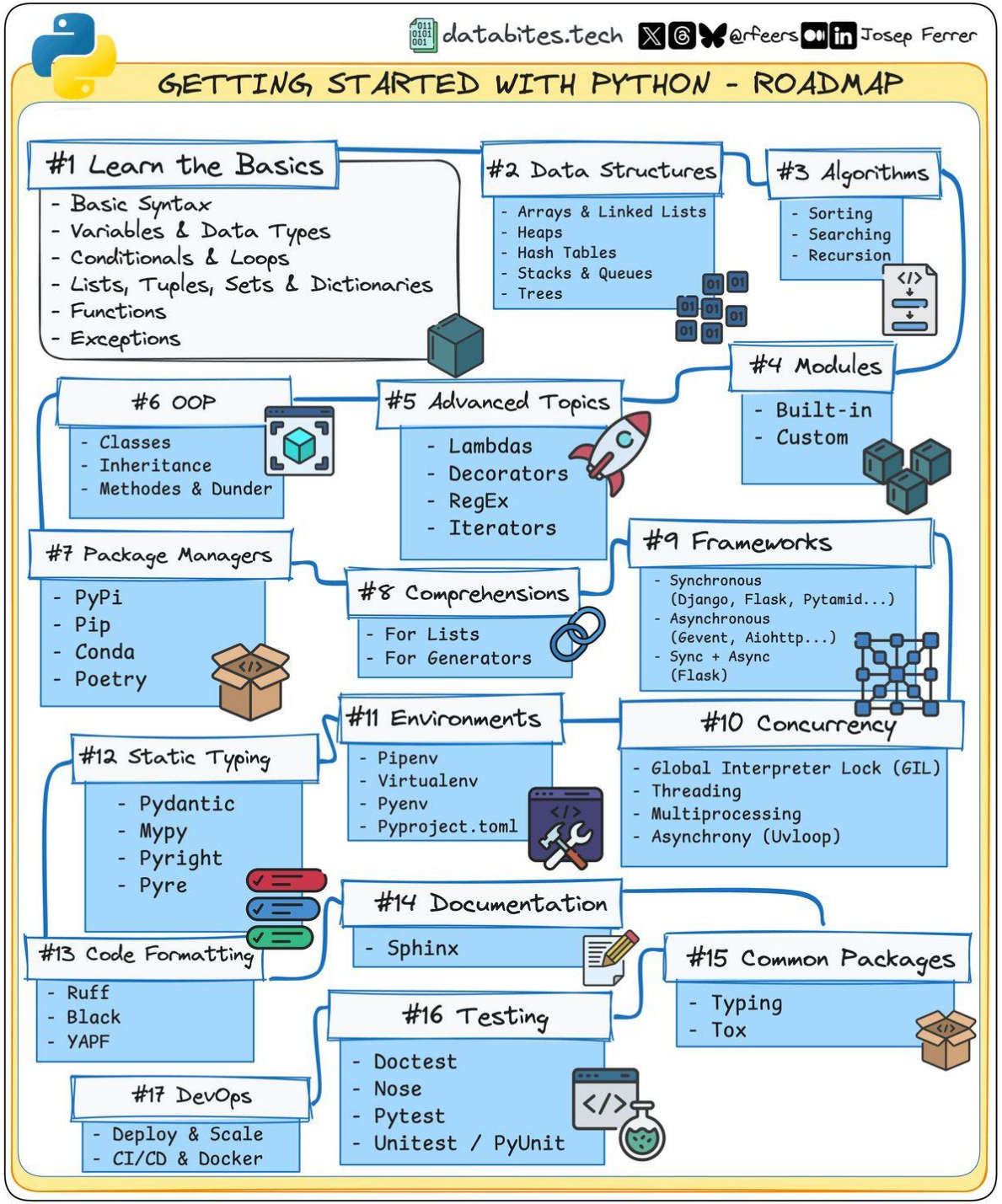

C++


Java

python™

Follow this on linkedin:

PEP 8



Is Python fast?

Libraries!

NUMPY

1. **Core:** Array processing library
   library
   ndrday (N-dim array)
3. **Functionality:** Fast numerical operations, Linear Algebra
4. **Performance:** Generally faster for for numerical computical computtions
5. **Use Case:** Scientific computing, raw data processing

VS

PANDAS

1. **Core:** Data manpulation
   library
   DraFrame, Series
3. **Functionality:** Data cleaning, analysis, SQL-like operations
4. **Performance:** Overhad for indexing, potentially slower for pure numerical tasks
5. **Use Case:** Data scienes analytics, ETL

The Python Package Index (PyPI) is a repository of software for the Python programming language.

PyPI helps you find and install software developed and shared by the Python community. Learn about installing packages.

Package authors use PyPI to distribute their software. Learn how to package your Python code for PyPI.

Good for:

1. **Data Exploration and Profiling**
2. **ETL Pipeline Prototyping**
3. **Data Quality Testing**
4. **Documentation and Knowledge Sharing**

```
pip install jupyter notebook ipykernel
```

Aspect	R	Python
Design Focus	Statistical analysis	General-purpose programming
Data as First-Class	Yes (vectors, data frames built-in)	No (needs pandas)
Missing Values	NA handled natively	NaN, None, needs careful handling
Vectorization	Default behavior	Requires numpy/pandas

The foundational language for data engineering. While not a general-purpose programming language, SQL is essential for querying, transforming, and managing data in relational databases and modern data warehouses.

Key Features:

- Declarative language for data manipulation
- Standardized across platforms (with dialects)
- Optimized for set-based operations
- Integrated with nearly all data tools

A JVM-based language that combines object-oriented and functional programming. Scala is the native language for Apache Spark, making it powerful for large-scale data processing.

Key Features:

- Type-safe with strong static typing
- Excellent performance on JVM
- Native Spark API (no translation overhead)
- Functional programming paradigms

The backbone of many big data technologies. Java powers Hadoop, Kafka, Elasticsearch, and numerous other data infrastructure tools.

Key Features:

- Platform independent (JVM)
- Excellent performance and memory management
- Vast ecosystem of libraries
- Strong typing and enterprise support

Python API for Apache Spark, allowing Python developers to leverage Spark's distributed computing capabilities.

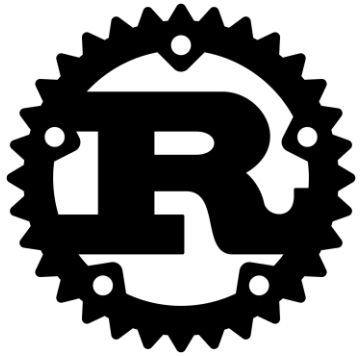
Key Features:

- Familiar Python syntax
- Access to Spark's full functionality
- Integration with Python data science libraries
- Slight performance overhead compared to Scala

A modern systems programming language gaining popularity in data engineering for building high-performance data pipelines and microservices.

Key Features:

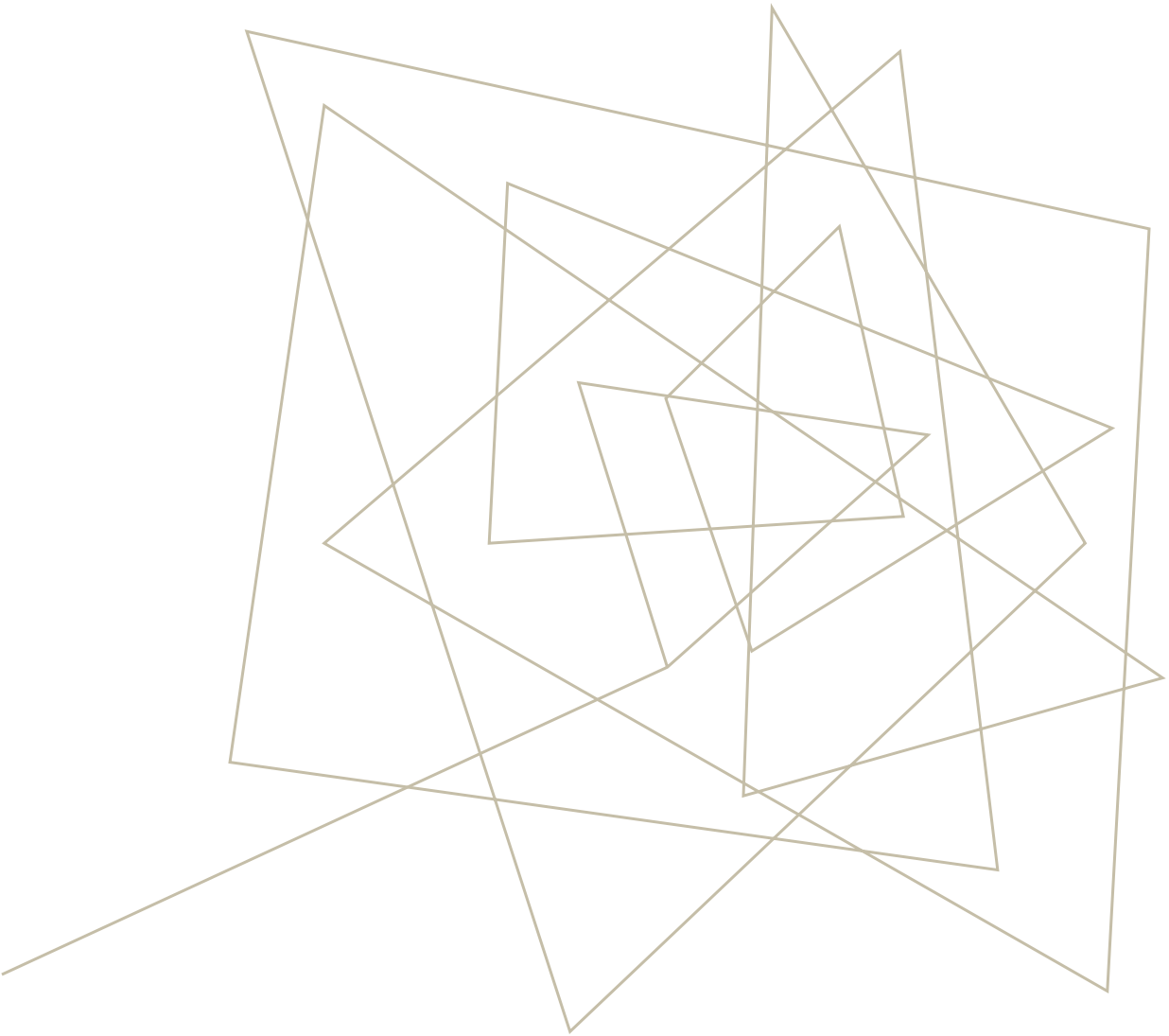
- Excellent concurrency with goroutines
- Fast compilation and execution
- Simple syntax
- Great for building data infrastructure tools



A systems programming language focusing on memory safety and performance, increasingly used for high-performance data processing tools.

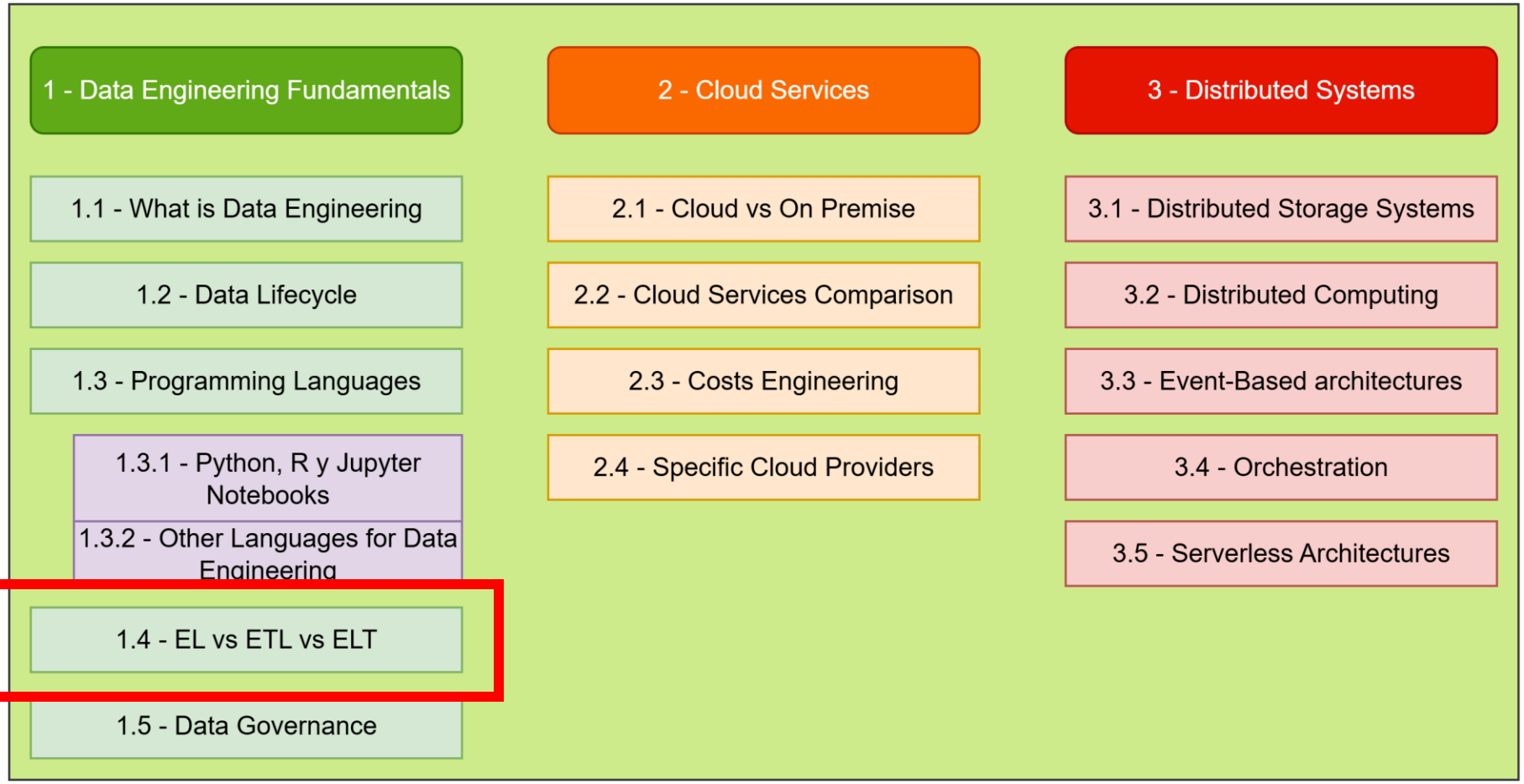
Key Features:

- Memory safety without garbage collection
- Zero-cost abstractions
- Excellent performance
- Growing ecosystem (Apache Arrow, DataFusion)



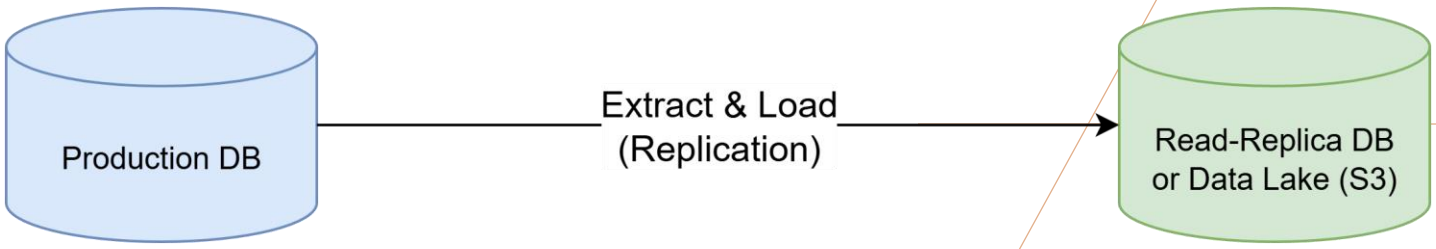
EL, ETL OR ELT

SUBJECT CURRICULUM



- **E - Extract:**
  - Reading and pulling data from one or more source systems.
  - *Sources:* Databases (SQL, NoSQL), APIs, SaaS applications (e.g., Salesforce), Log files.
- **T - Transform:**
  - The "business logic" step. Cleaning, enriching, and restructuring data.
  - *Examples:* Joining tables, aggregating values, masking PII, converting date formats.
- **L - Load:**
  - Writing the processed data into a target destination.
  - *Targets:* Data Warehouse, Data Lake, Data Mart.

EL

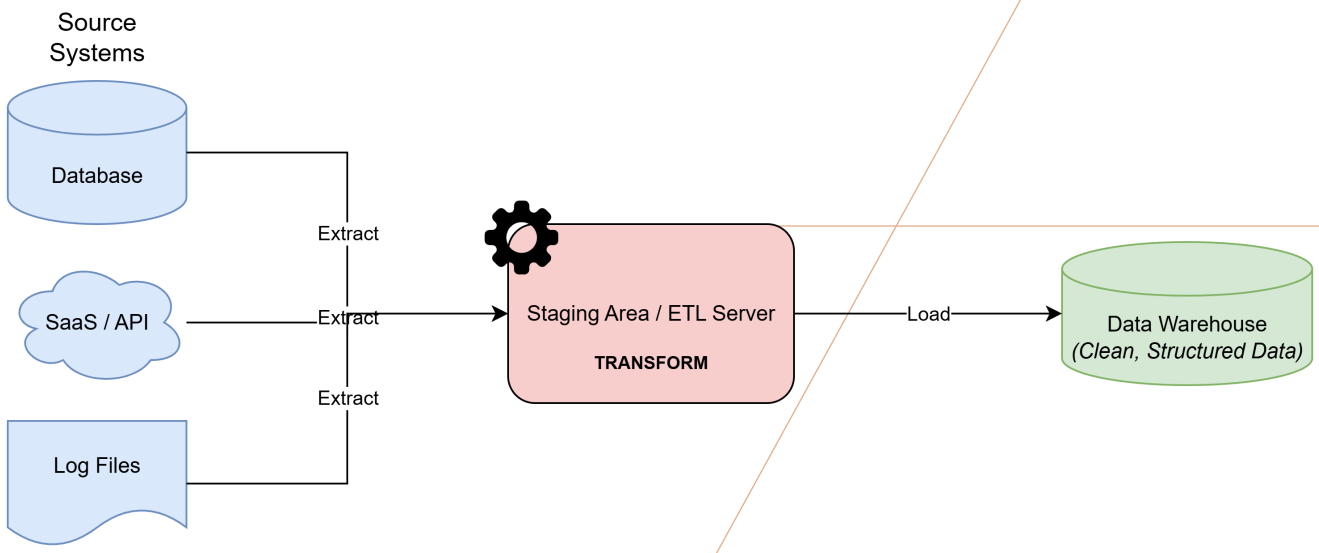


Database Replication: Creating a read-replica of a production database for analytical queries to avoid impacting production performance.

Data Lake Ingestion: Moving raw files (logs, images, JSON) into a data lake like S3 or ADLS for long-term storage or future processing.

Backup and Archiving: Simply moving data from one location to another for safekeeping.

CLASSIC ETL



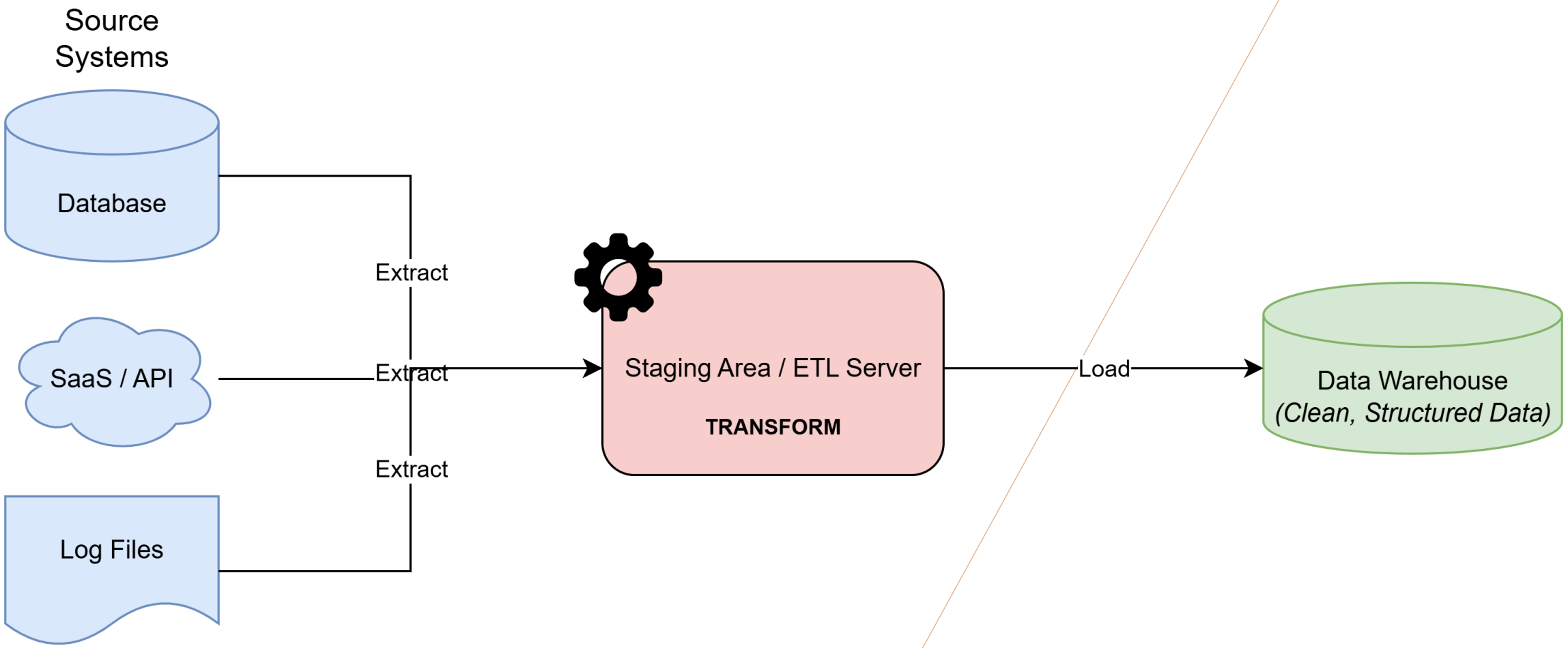
Extract data from sources.

Move it to a separate **staging area** or processing server.

Transform the data (clean, join, aggregate). This is often a resource-intensive step.

Load the clean, structured data into the target Data Warehouse.

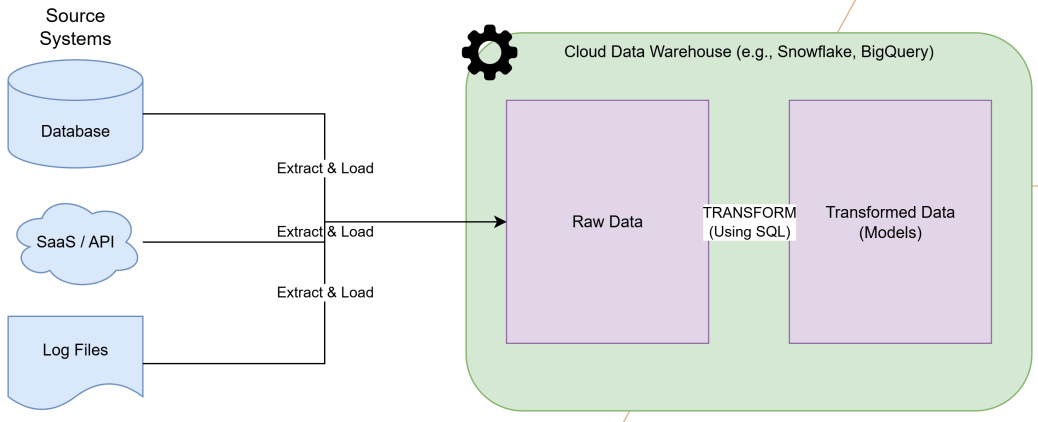
CLASSIC ETL


Source Systems
Database
SaaS / API
Log Files
Extract
Extract
Extract
Staging Area / ETL Server
TRANSFORM
Load
Data Warehouse
(Clean, Structured Data)

ETL: PROS & CONS

Pros	Cons
Mature & Well-Established: Tools are robust.	**Slow & Rigid:** Adding a new field can take weeks.
Compliance-Friendly: PII can be scrubbed before loading.	**Brittle:** Transformations can break with source changes.
Performs Complex Logic: Optimized for heavy transformations.	**High Maintenance:** Requires specialized skills to manage.
Reduces Load on Warehouse: Warehouse only handles queries.	**Not Ideal for Raw Data:** Raw data is discarded after transformation.

(MODERN) ELT

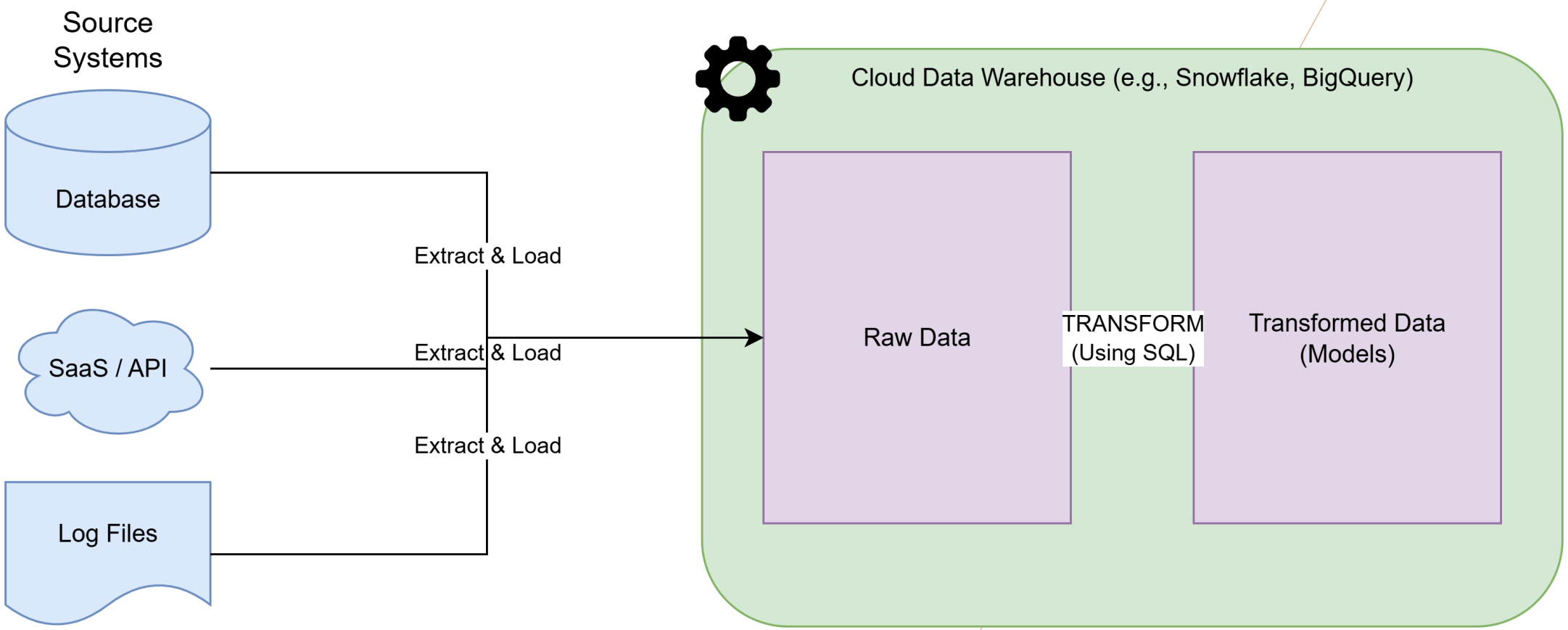


Extract data from sources.

Load the raw, untouched data directly into the target system (usually a cloud data warehouse like Snowflake, BigQuery, or Redshift).

Transform the data *inside* the warehouse using its powerful processing engine (e.g., using SQL).

(MODERN) ELT

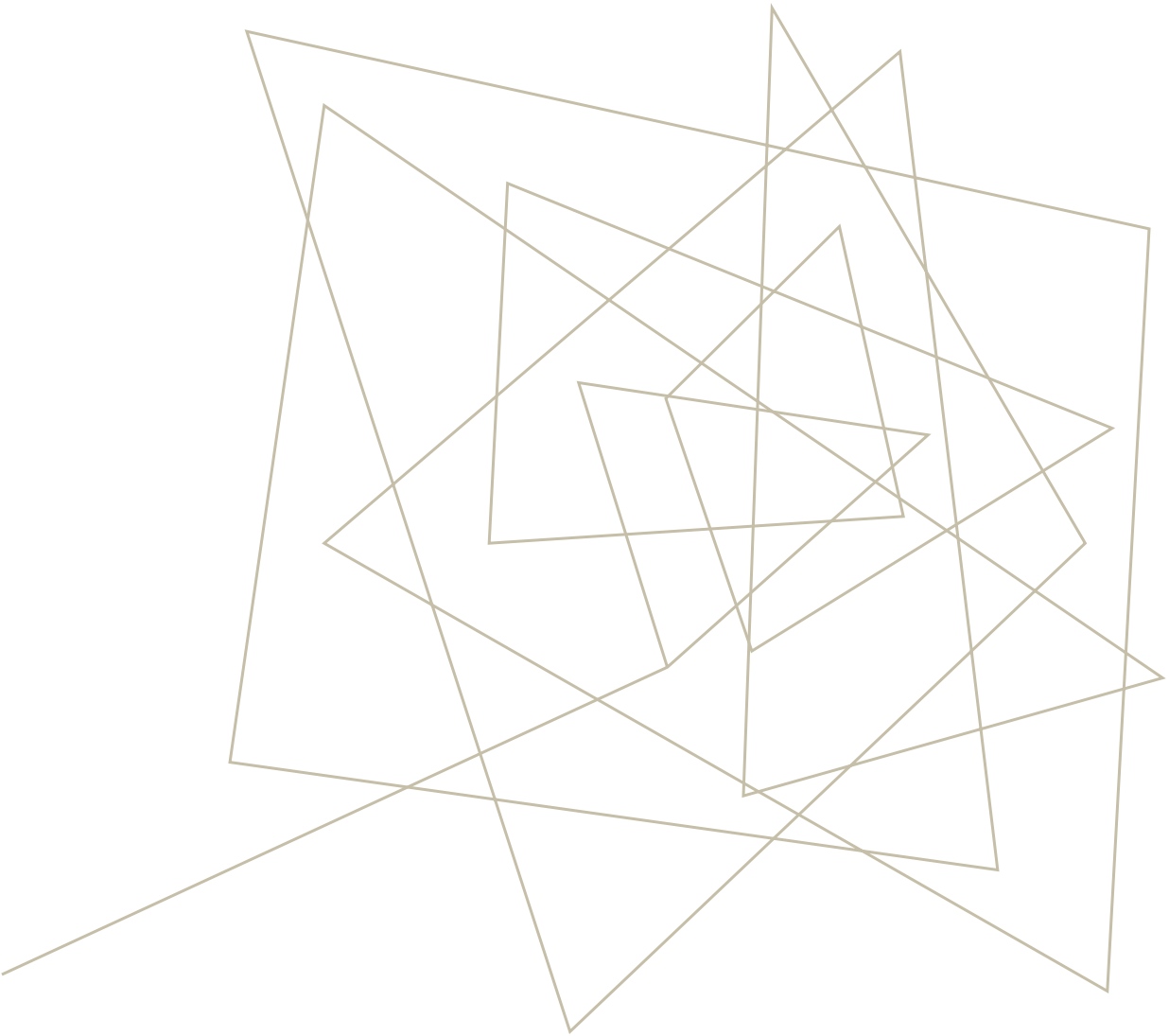

Source Systems
Database
SaaS / API
Log Files
Extract & Load
Extract & Load
Extract & Load
Cloud Data Warehouse (e.g., Snowflake, BigQuery)
Raw Data
TRANSFORM (Using SQL)
Transformed Data (Models)

ELT: PROS & CONS

Pros	Cons
Speed & Flexibility: Load raw data in minutes, not hours.	**Potential for a "Data Swamp":** Requires good governance.
Leverages Cloud Power: Uses the scalable compute of the warehouse.	**Cost Management is Key:** Pay-per-query models can get expensive.
"Schema-on-Read": No need to define structure upfront.	**PII is Loaded:** Sensitive data enters the warehouse raw.
All Data is Available: Analysts can access raw data for new insights.	**Transformation logic is in SQL:** Can become complex to manage.

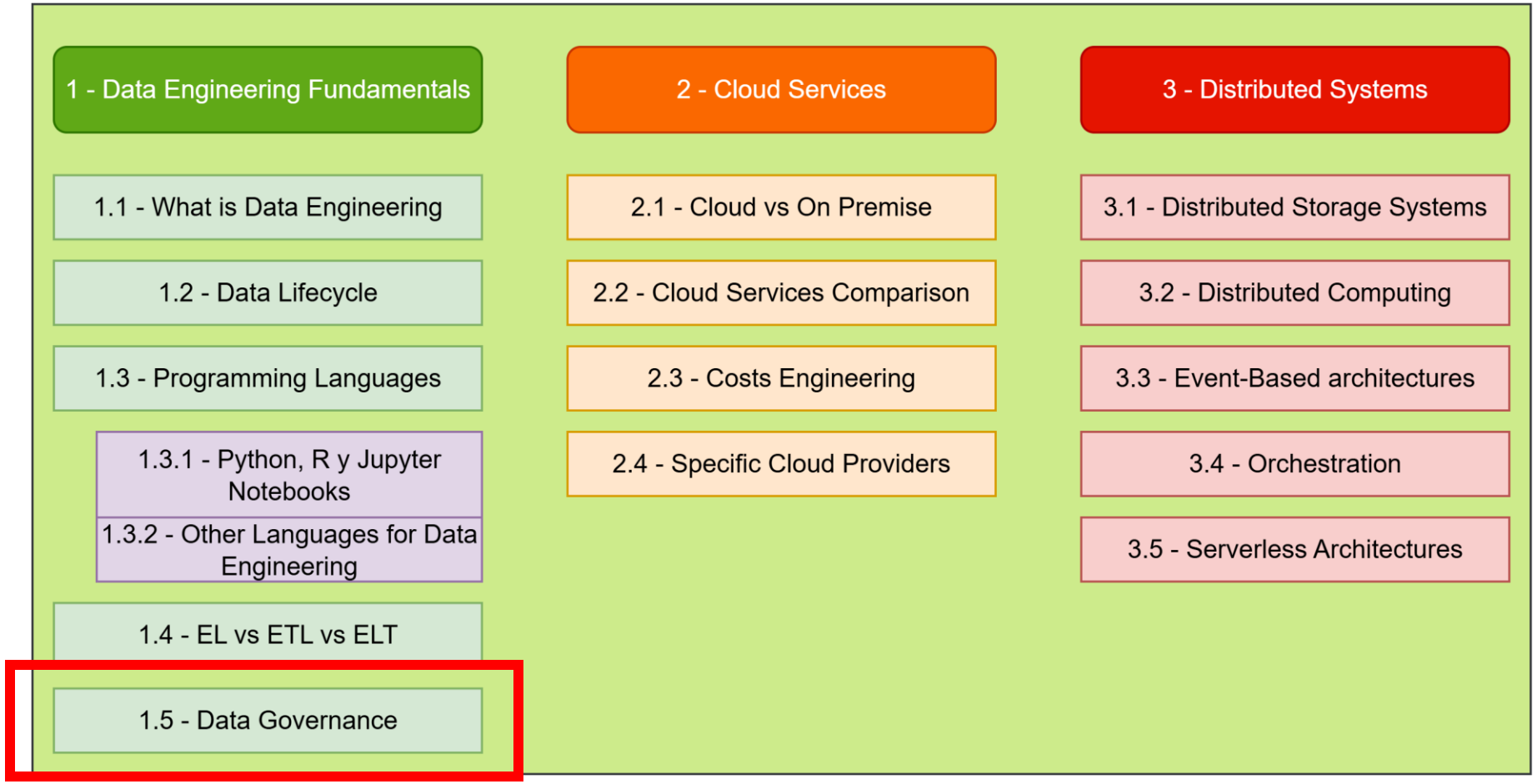
ETL VS ELT: COMPARISON

Feature	ETL (Extract, Transform, Load)	ELT (Extract, Load, Transform)
Transform Location	Staging Server / ETL Tool	Target Data Warehouse
Data Structure	Schema-on-Write (Structure first)	Schema-on-Read (Flexibility first)
Load Speed	Slower (waits for transform)	Faster (loads raw data immediately)
Data Availability	Only transformed data is available	Raw + Transformed data are available
Infrastructure	On-premise or Cloud	Primarily Cloud-based
Ideal Use Case	Structured data, strict compliance, complex pre-defined reports	Big data, semi-structured sources, exploratory analysis, speed
Example Tools	Informatica, Talend, SSIS	Fivetran, dbt, Snowflake, BigQuery



DATA GOVERNANCE

SUBJECT CURRICULUM



WHAT IS DATA GOVERNANCE

- Formal management of data assets throughout their lifecycle
- Set of processes, policies, and standards ensuring data quality, security, and compliance
- **Key Objectives:**
  - Data Quality & Integrity
  - Data Security & Privacy
  - Regulatory Compliance
  - Data Accessibility & Usability

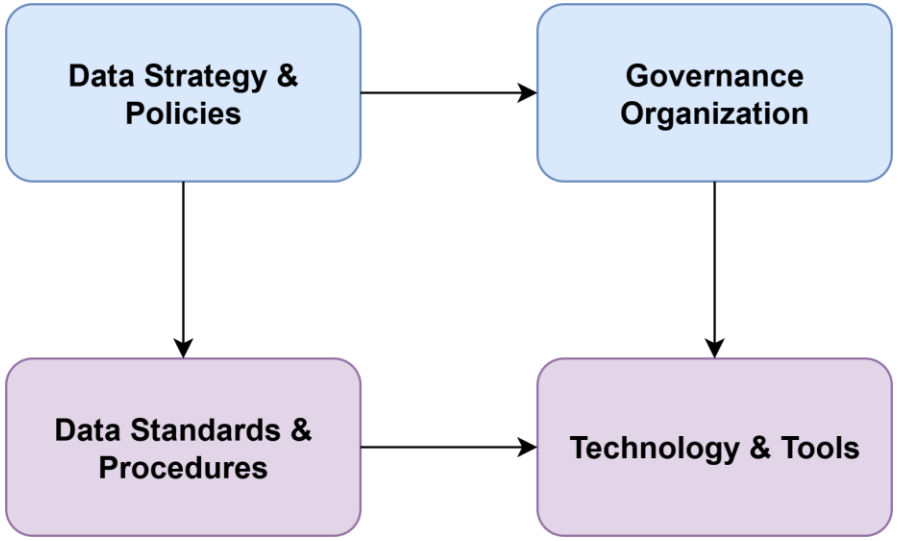
WHY SHOULD CARE

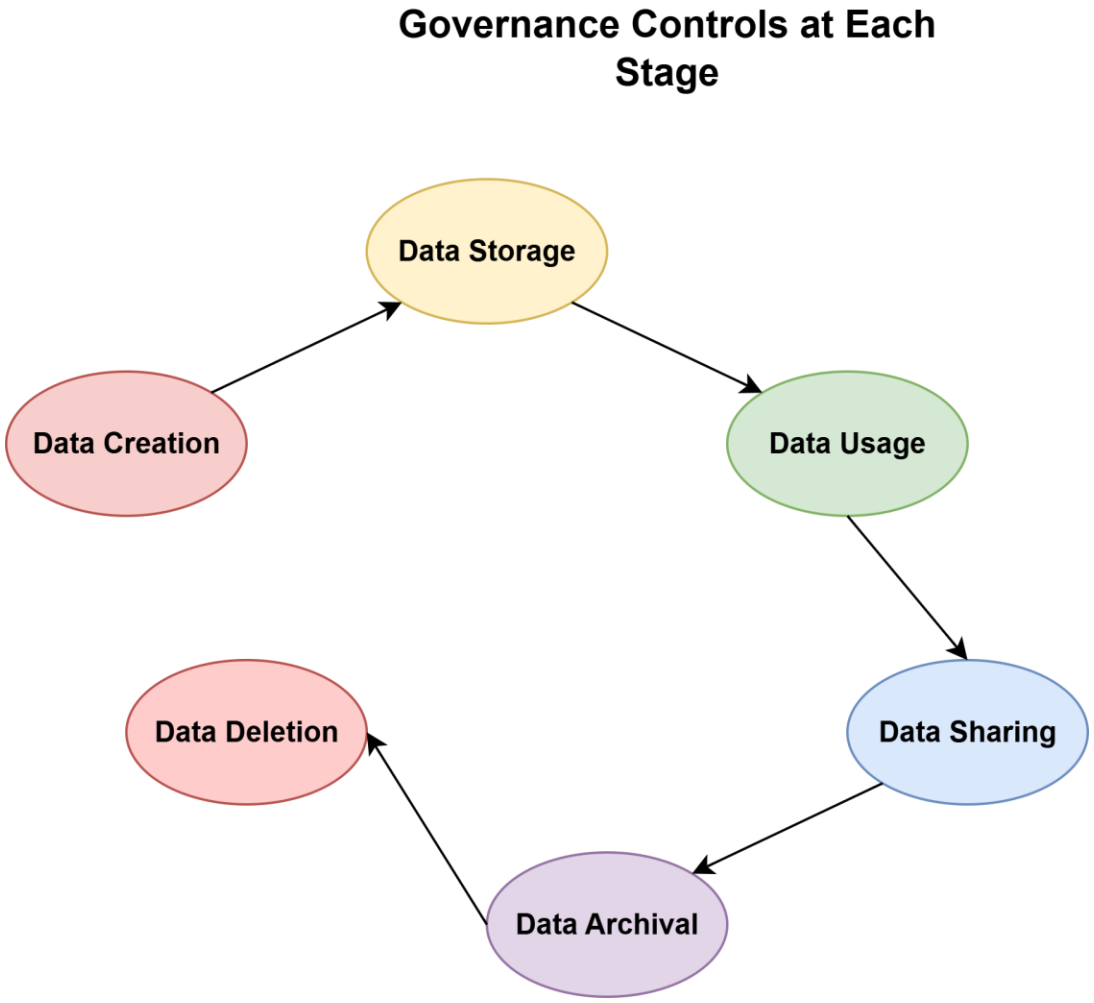
- **Business Impact:**
  - Poor data costs organizations $15M annually (average)
  - 40% of business initiatives fail due to data quality issues
- **Engineering Challenges:**
  - Data pipeline failures due to schema changes
  - Inconsistent data formats across systems
  - Security vulnerabilities in data flows
  - Compliance violations (GDPR, CCPA)

DATA GOVERNANCE CORE PILLARS

- **Data Quality Management**
  - Accuracy, Completeness, Consistency, Timeliness
- **Data Security & Privacy**
  - Access controls, Encryption, PII protection
- **Data Lifecycle Management**
  - Creation, Storage, Usage, Archival, Deletion
- **Metadata Management**
  - Data lineage, Cataloging, Documentation

DATA GOVERNANCE FRAMEWORK

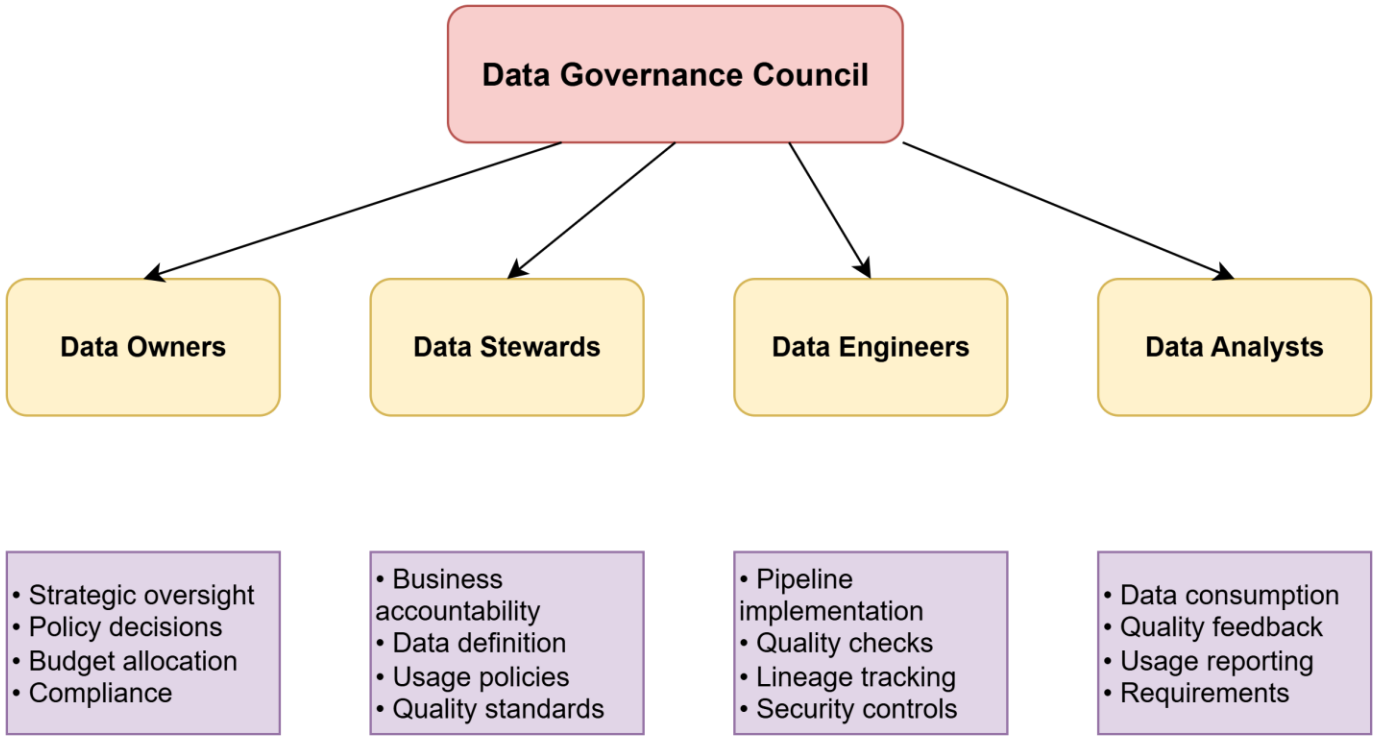




KEY ROLES

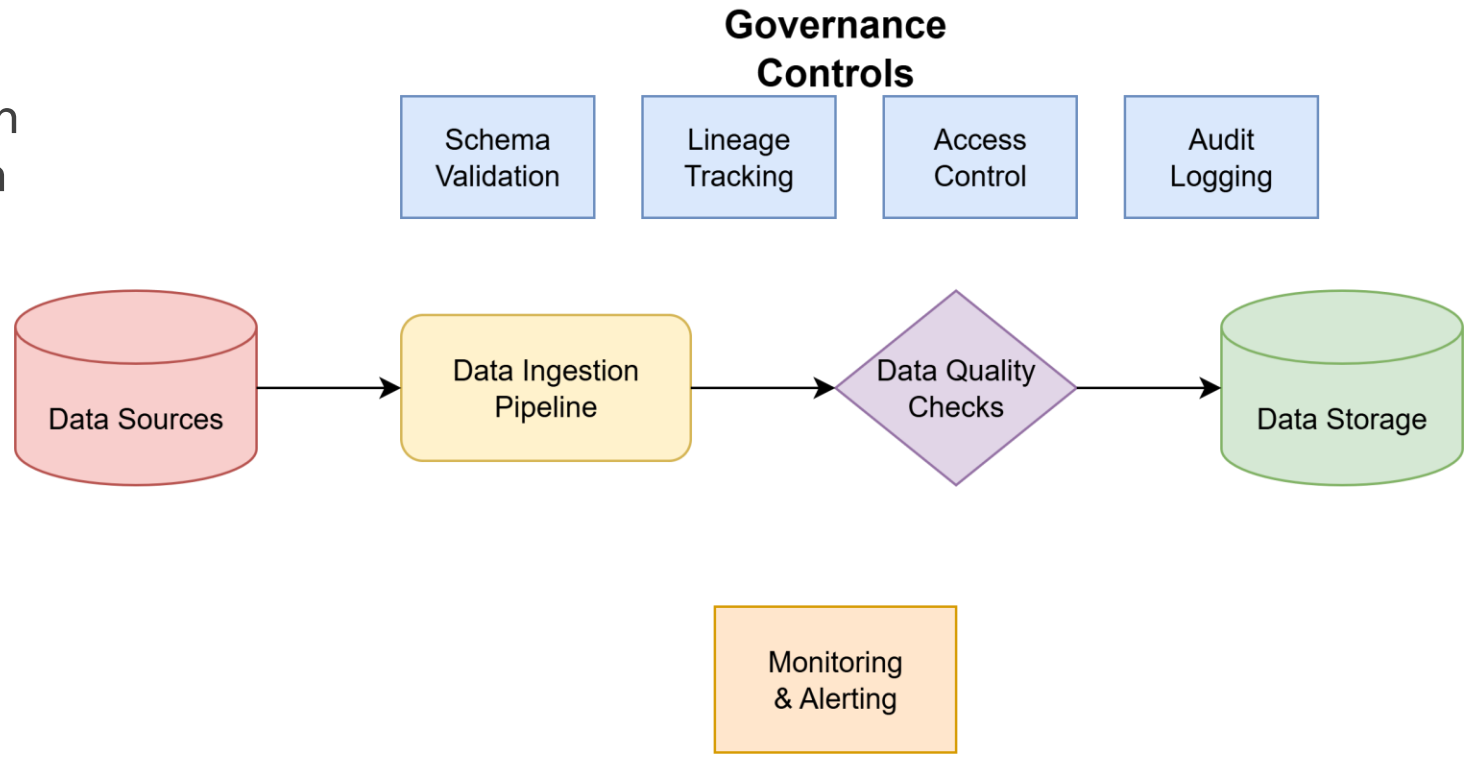
- **Data Governance Council**
  - Strategic oversight and policy decisions
- **Data Owners**
  - Business accountability for specific datasets
- **Data Stewards**
  - Day-to-day data management and quality
- **Data Engineers**
  - Implement governance controls in pipelines
  - Ensure data lineage tracking
  - Maintain data quality checks

KEY ROLES


Data Governance Council
Data Owners
Data Stewards
Data Engineers
Data Analysts
• Strategic oversight
• Policy decisions
• Budget allocation
• Compliance
• Business accountability
• Data definition
• Usage policies
• Quality standards
• Pipeline implementation
• Quality checks
• Lineage tracking
• Security controls
• Data consumption
• Quality feedback
• Usage reporting
• Requirements

GOVERNANCE TASKS FORE DATA ENGINEERING

- **Pipeline Development:**
  - Schema validation and evolution
  - Data quality checks at ingestion
  - Error handling and monitoring
- **Infrastructure:**
  - Access control implementation
  - Audit logging
  - Data encryption in transit/rest
- **Documentation:**
  - Data lineage mapping
  - Pipeline documentation
  - Incident response procedures



TOOLS AND TECHNOLOGIES

- **Data Catalogs:**
  - Apache Atlas, DataHub, Collibra
- **Data Quality:**
  - Great Expectations, Deequ, Monte Carlo
- **Lineage & Metadata:**
  - Apache Atlas, DataHub, Airflow
- **Access Control:**
  - Apache Ranger, AWS IAM, Databricks Unity Catalog
- **Monitoring:**
  - DataDog, Prometheus, custom dashboards

CHALLENGES

- **Technical Challenges:**
  - Legacy system integration
  - Real-time governance at scale
  - Cross-platform data lineage
- **Organizational Challenges:**
  - Resistance to change
  - Lack of data literacy
  - Siloed data ownership
- **Solutions:**
  - Start small with pilot projects
  - Automate governance controls
  - Provide training and support

BEST PRACTICES

- **Build Governance into Pipelines**
  - Automated data quality tests
  - Schema validation
  - Lineage tracking
- **Implement Data Contracts**
  - Clear SLAs between teams
  - Versioned schemas
  - Breaking change notifications
- **Monitor & Alert**
  - Data quality metrics
  - Pipeline health dashboards
  - Anomaly detection

CASE STUDY: NETFLIX

- **Challenge:**
  - Massive data volumes from global streaming
  - Multiple engineering teams
  - Regulatory compliance across regions
- **Solution:**
  - Centralized data platform with governance
  - Automated data quality monitoring
  - Self-service data access with controls
- **Results:**
  - 40% reduction in data quality incidents
  - Improved compliance posture
  - Faster time-to-insight for analytics

GDPR

The **General Data Protection Regulation (GDPR)** is a comprehensive data protection law that came into effect on May 25, 2018. It applies to:

- All organizations processing personal data of EU residents.
- Any organization with operations in the EU.
- Any organization offering goods/services to EU residents: Even non-EU companies must comply if they handle EU citizens' data.

GDPR: KEY DEFINITIONS

Personal Data

Any information relating to an identified or identifiable natural person, including:

Direct identifiers: Name, ID numbers, email addresses

Indirect identifiers: IP addresses, location data, online identifiers

Special categories: Health, biometric, genetic, political opinions, religious beliefs

Pseudonymized data: Still considered personal data under GDPR

Data Controller vs. Data Processor

Controller: Determines purposes and means of processing (your organization)

Processor: Processes data on behalf of controller (cloud providers, vendors)

GDPR: PRINCIPLES

Technical Measures

✓ Encryption at rest and in transit
✓ Pseudonymization where possible
✓ Access controls and authentication
✓ Data backup and recovery
✓ Automated data deletion capabilities

Organizational Measures

✓ Privacy policies and procedures
✓ Staff training on GDPR
✓ Data Protection Officer (if required)
✓ Vendor agreements (Data Processing Agreements)
✓ Incident response procedures

Red Flags to Avoid

❌ Processing without legal basis
❌ Keeping data longer than necessary
❌ Insufficient security measures
❌ No process for data subject requests
❌ Sharing data without proper agreements
❌ No documentation of processing activities
❌ Ignoring breach notification requirements
❌ Invalid consent mechanisms

Data Subject Request Procedures

•Implement systems to verify identity
•Create processes to search across all systems
•Establish workflows for each type of request
•Set up tracking and response mechanisms

GDPR: WRAP UP

1. **GDPR applies to most organizations** handling EU residents' data
2. **Personal data definition is broad** - includes identifiers and pseudonymized data
3. **Must have legal basis** for all personal data processing
4. **Individual rights are extensive** - prepare systems to respond
5. **Security and privacy by design** are mandatory
6. **Documentation is crucial** for demonstrating compliance
7. **Penalties are significant** - up to 4% of global revenue
8. **Ongoing compliance** - not a one-time project

Remember: GDPR is about protecting individuals' fundamental rights while enabling legitimate business operations. Focus on building trust through transparent, secure, and respectful data practices.

DATA GOVERNANCE SPEED DATING!

Speed Dating Exercise